

NNDL Assignment No. 4 — Unit 4

Advanced DCGAN-Based Synthetic Fashion Image Generation with Quantitative Evaluation

Course: Neural Network and Deep Learning

Course Code: AIM5031C

Program: M.Tech Computer Engineering (Artificial Intelligence and Machine Learning)

Semester: 1st

Selected Model

Deep Convolutional Generative Adversarial Network (DCGAN)

Dataset

Fashion-MNIST

Framework

Python, TensorFlow and Keras

Platform

Jupyter Notebook

2. Objective

The objective of this mini-capstone project is to design and implement a Deep Convolutional Generative Adversarial Network (DCGAN) for generating synthetic fashion images using the Fashion-MNIST dataset.

The project aims to:

- Understand the working principle of Generative Adversarial Networks (GANs).
- Implement a Generator and a Discriminator using deep convolutional neural network architectures.
- Train the Generator and Discriminator using adversarial learning.
- Generate new synthetic images that resemble the images in the Fashion-MNIST dataset.
- Analyze the training behavior of the Generator and Discriminator.
- Visualize generated images at different stages of training.

- Evaluate the quality of generated images using visual and quantitative evaluation techniques.
- Analyze the limitations and performance of the developed DCGAN model.

3. Problem Statement

Generative models are an important area of deep learning that can learn the underlying distribution of data and generate new synthetic samples.

In this project, a Deep Convolutional Generative Adversarial Network (DCGAN) is developed to generate synthetic fashion images based on the Fashion-MNIST dataset.

The system consists of two neural networks:

1. **Generator** – generates synthetic images from random noise.
2. **Discriminator** – distinguishes between real images from the dataset and synthetic images generated by the Generator.

Both networks are trained adversarially so that the Generator gradually learns to produce realistic-looking images that can be difficult for the Discriminator to distinguish from real images.

The project also analyzes the training process and evaluates the quality of generated images using visual and quantitative evaluation methods.

4. Introduction

Generative Artificial Intelligence focuses on developing models that can learn patterns and distributions from existing data and generate new data samples.

A Generative Adversarial Network (GAN) is a deep learning framework consisting of two competing neural networks: a Generator and a Discriminator.

The Generator attempts to create realistic synthetic data, while the Discriminator attempts to distinguish real data from generated data. Through this adversarial training process, the Generator gradually improves its ability to generate realistic samples.

GANs have applications in areas such as:

- Image generation
- Image-to-image translation
- Data augmentation
- Synthetic data generation
- Image restoration
- Computer vision

- Creative content generation
- Research and simulation

In this project, a Deep Convolutional Generative Adversarial Network (DCGAN) is implemented for synthetic fashion image generation using the Fashion-MNIST dataset.

The project goes beyond basic GAN implementation by analyzing training losses, generated-image progression, visual quality, and quantitative evaluation.

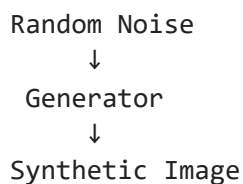
5. Generative Adversarial Network (GAN) Concept

A Generative Adversarial Network (GAN) consists of two neural networks:

5.1 Generator

The Generator creates synthetic data from a random noise vector.

The Generator receives random noise as input and transforms it into a synthetic image.



5.2 Discriminator

The Discriminator receives both real and generated images and attempts to determine whether an image is real or generated.



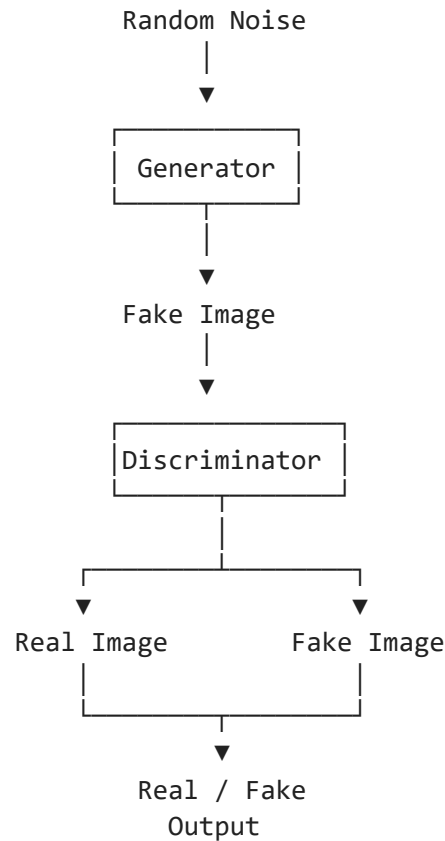
5.3 Adversarial Training

The Generator and Discriminator have competing objectives.

The Generator attempts to generate images that appear real. The Discriminator attempts to correctly identify real and generated images.

During training, both networks are updated repeatedly. The Generator learns from the feedback provided by the Discriminator and gradually improves the quality of generated images.

Basic GAN Process



Section 6 — Why DCGAN?

This section will make the project stronger because we need to explain **why we selected DCGAN instead of a simple GAN.**

Markdown Cell

6. Why DCGAN?

A basic GAN can use fully connected neural networks, but image data contains important spatial information. Convolutional neural networks are well suited for learning spatial patterns and visual features.

A Deep Convolutional Generative Adversarial Network (DCGAN) combines GAN-based adversarial learning with convolutional neural network architectures.

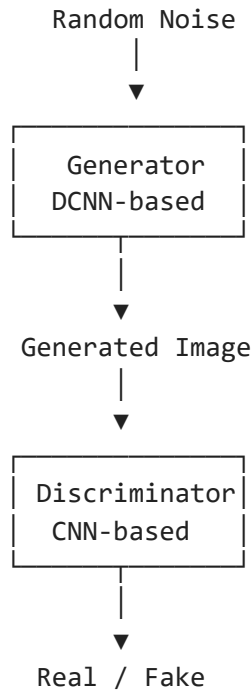
In this project, DCGAN is selected because:

- It is specifically designed for image generation tasks.
- Convolutional layers can learn spatial features from images.

- The Generator can use transposed convolutional layers to progressively construct images.
- The Discriminator can use convolutional layers to identify visual patterns.
- It provides a suitable architecture for generating Fashion-MNIST images.
- It allows us to analyze both image quality and adversarial training behavior.

DCGAN Conceptual Architecture

```
```text
```



---

## Section 7 — Project Workflow

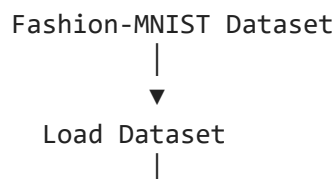
This is the main workflow of the entire notebook.

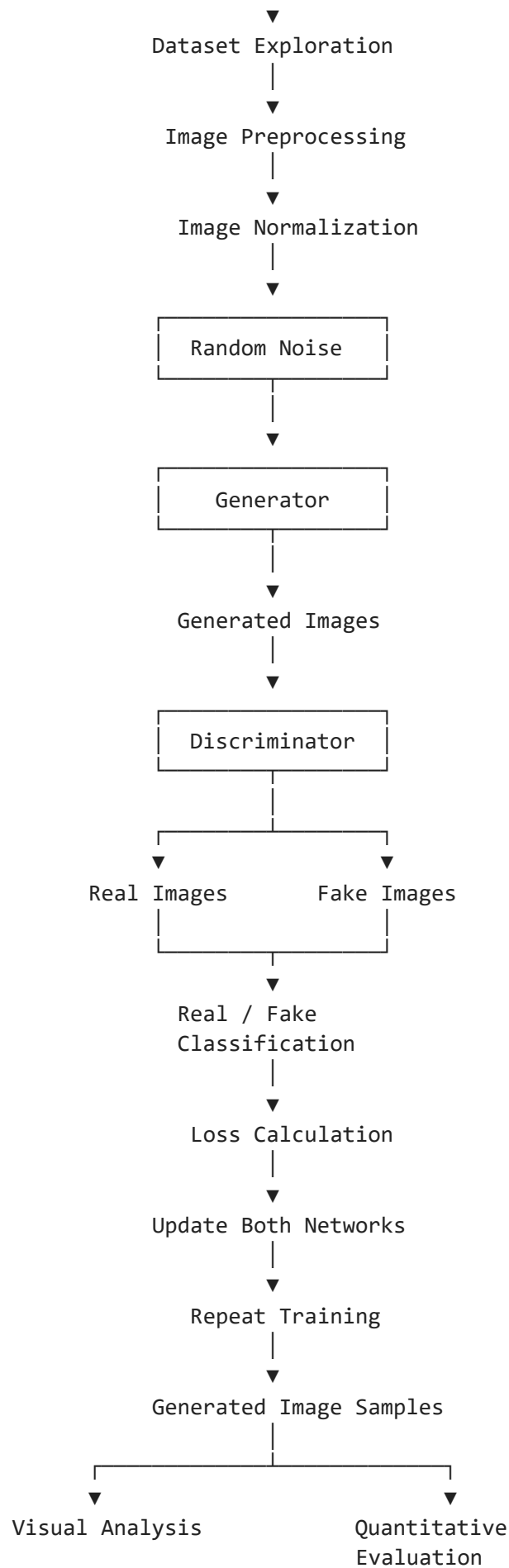
### Markdown Cell

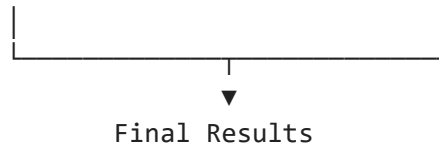
#### ## 7. Project Workflow

The complete project follows the workflow below:

```
```text
```







Section 8 — Dataset Description

Markdown Cell

8. Dataset Description

Dataset Name

****Fashion-MNIST****

Fashion-MNIST is an image dataset containing grayscale images of fashion-related items. It is commonly used for machine learning and deep learning experiments involving image data.

Dataset Characteristics

- Image type: Grayscale
- Image size: 28 × 28 pixels
- Number of classes: 10
- Each image represents a fashion item.
- The dataset can be used directly through TensorFlow/Keras dataset utilities.

Fashion Categories

The dataset contains the following categories:

1. T-shirt / Top
2. Trouser
3. Pullover
4. Dress
5. Coat
6. Sandal
7. Shirt
8. Sneaker
9. Bag
10. Ankle Boot

Role of the Dataset in This Project

In this project, the Fashion-MNIST images are used as real training samples for the Discriminator.

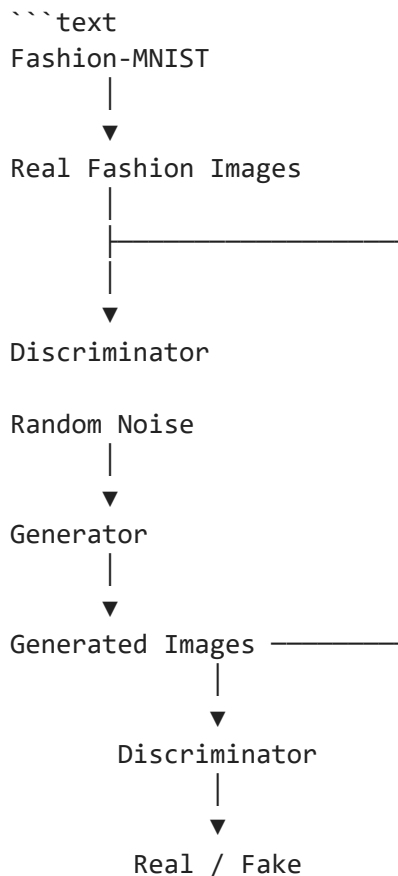
The Generator does not directly receive real images as input. Instead, it receives random noise and attempts to generate synthetic fashion images.

The Discriminator receives:

- Real Fashion-MNIST images
- Generated images from the Generator

and learns to distinguish between the two.

Dataset Flow



```

In [2]: # =====
# Section 9: Import Required Libraries
# =====

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# Set a fixed random seed so that experiments are more reproducible
SEED = 42
  
```

```
np.random.seed(SEED)
tf.random.set_seed(SEED)

print("All required libraries imported successfully.")
```

All required libraries imported successfully.

```
In [3]: # Check the installed TensorFlow version
print("TensorFlow Version:", tf.__version__)
```

TensorFlow Version: 2.21.0

```
In [4]: # =====
# Check whether a GPU is available for faster training
# =====

gpus = tf.config.list_physical_devices("GPU")

if gpus:
    print("GPU is available.")
    print("Number of GPUs:", len(gpus))
    for gpu in gpus:
        print(gpu)
else:
    print("No GPU detected. Training will use the CPU.")
```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

No GPU detected. Training will use the CPU.

10. Load Fashion-MNIST Dataset

The Fashion-MNIST dataset is loaded using the TensorFlow/Keras dataset utility. The dataset contains grayscale fashion images of size 28×28 pixels.

For GAN training, the image data will be used as real samples for the Discriminator. The Generator will learn to create synthetic images that resemble these real samples.

```
In [5]: # =====
# Section 10: Load Fashion-MNIST Dataset
# =====

# Load the Fashion-MNIST dataset from TensorFlow/Keras
(X_train, y_train), (X_test, y_test) = keras.datasets.fashion_mnist.load_data()

print("Fashion-MNIST dataset loaded successfully.")
print("Training images shape:", X_train.shape)
print("Training labels shape:", y_train.shape)
print("Testing images shape:", X_test.shape)
print("Testing labels shape:", y_test.shape)
```

```
Fashion-MNIST dataset loaded successfully.  
Training images shape: (60000, 28, 28)  
Training labels shape: (60000,)  
Testing images shape: (10000, 28, 28)  
Testing labels shape: (10000,)
```

```
In [6]: # Display basic information about the dataset  
  
print("Number of training images:", len(X_train))  
print("Number of testing images:", len(X_test))  
print("Image height:", X_train.shape[1])  
print("Image width:", X_train.shape[2])  
print("Number of classes:", len(np.unique(y_train)))
```

```
Number of training images: 60000  
Number of testing images: 10000  
Image height: 28  
Image width: 28  
Number of classes: 10
```

11. Dataset Exploration

Before training the DCGAN, the dataset is explored to understand the image structure and the distribution of the available classes.

Although the GAN does not require class labels for basic unconditional image generation, the labels are useful for understanding and visualizing the original Fashion-MNIST dataset.

```
In [7]: # Fashion-MNIST class names  
class_names = [  
    "T-shirt/Top",  
    "Trouser",  
    "Pullover",  
    "Dress",  
    "Coat",  
    "Sandal",  
    "Shirt",  
    "Sneaker",  
    "Bag",  
    "Ankle Boot"  
]  
  
print("Fashion-MNIST Classes:")  
for index, name in enumerate(class_names):  
    print(f"{index}: {name}")
```

Fashion-MNIST Classes:

- 0: T-shirt/Top
- 1: Trouser
- 2: Pullover
- 3: Dress
- 4: Coat
- 5: Sandal
- 6: Shirt
- 7: Sneaker
- 8: Bag
- 9: Ankle Boot

12. Visualize Real Images

The following visualization displays sample images from the Fashion-MNIST training dataset.

These images represent the real samples that the Discriminator will learn to distinguish from images generated by the Generator.

```
In [8]: # =====  
# Visualize sample images from the real training dataset  
# =====  
  
plt.figure(figsize=(10, 6))  
  
for i in range(12):  
    plt.subplot(3, 4, i + 1)  
  
    # Display the grayscale image  
    plt.imshow(X_train[i], cmap="gray")  
  
    # Display the corresponding class name  
    plt.title(class_names[y_train[i]])  
  
    plt.axis("off")  
  
plt.tight_layout()  
plt.show()
```



```
In [9]: # Check the minimum and maximum pixel values
print("Minimum pixel value:", X_train.min())
print("Maximum pixel value:", X_train.max())
```

```
Minimum pixel value: 0
Maximum pixel value: 255
```

13. Data Preprocessing

The original Fashion-MNIST images contain pixel values in the range 0 to 255.

For stable GAN training, the images are converted to floating-point values and normalized to the range $[-1, 1]$.

The output activation of the Generator will use `tanh`, which also produces values in the range $[-1, 1]$. Therefore, both real and generated images will use the same numerical range during adversarial training.

```
In [10]: # =====
# Normalize image pixel values from [0, 255] to [-1, 1]
# =====

X_train = X_train.astype("float32")

X_train = (X_train - 127.5) / 127.5

print("Minimum pixel value after normalization:", X_train.min())
print("Maximum pixel value after normalization:", X_train.max())
```

Minimum pixel value after normalization: -1.0

Maximum pixel value after normalization: 1.0

```
In [11]: # Add a channel dimension for grayscale images
# Shape changes from (samples, 28, 28) to (samples, 28, 28, 1)

X_train = np.expand_dims(X_train, axis=-1)

print("Training data shape after adding channel:", X_train.shape)
```

Training data shape after adding channel: (60000, 28, 28, 1)

14. Prepare Training Dataset

The preprocessed images are converted into a TensorFlow dataset. Batching and shuffling are used to efficiently provide real images during GAN training.

A fixed batch size is used so that the Generator and Discriminator can process multiple images during each training step.

```
In [12]: # =====
# Prepare the TensorFlow dataset for GAN training
# =====

BATCH_SIZE = 128
BUFFER_SIZE = 60000

train_dataset = (
    tf.data.Dataset.from_tensor_slices(X_train)
    .shuffle(BUFFER_SIZE, seed=SEED)
    .batch(BATCH_SIZE, drop_remainder=True)
    .prefetch(tf.data.AUTOTUNE)
)

print("Training dataset prepared successfully.")
print("Batch size:", BATCH_SIZE)
```

Training dataset prepared successfully.

Batch size: 128

15. Define Generator

The Generator is responsible for creating synthetic fashion images from a random noise vector.

The model starts with a low-dimensional random noise vector and progressively transforms it into a 28×28 grayscale image using dense and transposed convolutional layers.

Batch normalization is used to improve training stability, while ReLU activation functions are used in the intermediate layers. The final layer uses the `tanh` activation function because the training images were normalized to the range $[-1, 1]$.

```
In [13]: # =====
# Section 15: Define Generator
# =====

# Dimension of the random noise vector given to the Generator
LATENT_DIM = 100

print("Latent dimension:", LATENT_DIM)
```

Latent dimension: 100

```
In [14]: # =====
# Build the DCGAN Generator
# =====

def build_generator():
    """
    Creates the Generator network.

    Input:
        Random noise vector of size LATENT_DIM.

    Output:
        Generated grayscale image of shape 28 x 28 x 1.
    """

    model = keras.Sequential(name="DCGAN_Generator")

    # Start with a random noise vector
    model.add(
        layers.Input(shape=(LATENT_DIM,))
    )

    # Project the noise vector into a small spatial feature map
    model.add(
        layers.Dense(7 * 7 * 128, use_bias=False)
    )
    model.add(
        layers.BatchNormalization()
    )
    model.add(
        layers.ReLU()
    )

    # Reshape into a 7 x 7 feature map with 128 channels
    model.add(
        layers.Reshape((7, 7, 128))
    )

    # Upsample: 7 x 7 -> 14 x 14
    model.add(
        layers.Conv2DTranspose(
            64,
            kernel_size=5,
            strides=2,
            padding="same",
```

```

        use_bias=False
    )
)
model.add(
    layers.BatchNormalization()
)
model.add(
    layers.ReLU()
)

# Upsample: 14 x 14 -> 28 x 28
model.add(
    layers.Conv2DTranspose(
        32,
        kernel_size=5,
        strides=2,
        padding="same",
        use_bias=False
    )
)
model.add(
    layers.BatchNormalization()
)
model.add(
    layers.ReLU()
)

# Final layer produces a single-channel grayscale image
model.add(
    layers.Conv2DTranspose(
        1,
        kernel_size=5,
        strides=1,
        padding="same",
        activation="tanh"
    )
)

return model

# Create the Generator
generator = build_generator()

print("Generator created successfully.")

```

Generator created successfully.

```
In [15]: # Display the Generator architecture
generator.summary()
```

Model: "DCGAN_Generator"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 6272)	627,200
batch_normalization (BatchNormalization)	(None, 6272)	25,088
re_lu (ReLU)	(None, 6272)	0
reshape (Reshape)	(None, 7, 7, 128)	0
conv2d_transpose (Conv2DTranspose)	(None, 14, 14, 64)	204,800
batch_normalization_1 (BatchNormalization)	(None, 14, 14, 64)	256
re_lu_1 (ReLU)	(None, 14, 14, 64)	0
conv2d_transpose_1 (Conv2DTranspose)	(None, 28, 28, 32)	51,200
batch_normalization_2 (BatchNormalization)	(None, 28, 28, 32)	128
re_lu_2 (ReLU)	(None, 28, 28, 32)	0
conv2d_transpose_2 (Conv2DTranspose)	(None, 28, 28, 1)	801

Total params: 909,473 (3.47 MB)

Trainable params: 896,737 (3.42 MB)

Non-trainable params: 12,736 (49.75 KB)

```
In [16]: # =====
# Test the Generator using random noise
# =====

# Generate random latent vectors
test_noise = tf.random.normal(
    shape=(4, LATENT_DIM),
    seed=SEED
)

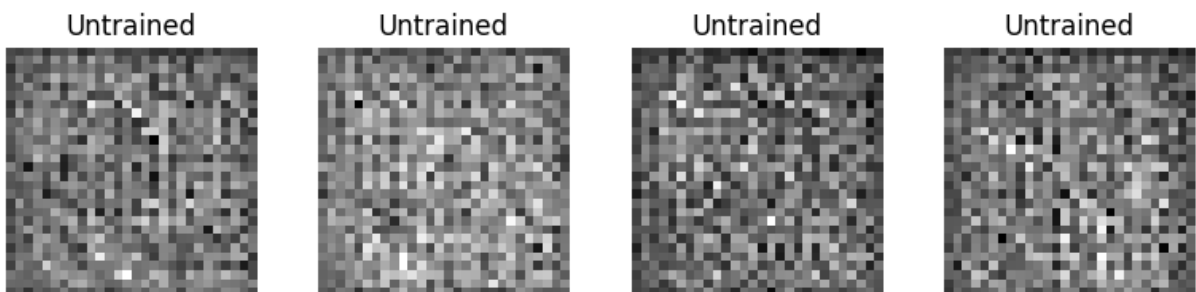
# Generate four synthetic images
generated_test_images = generator(test_noise, training=False)

print("Input noise shape:", test_noise.shape)
print("Generated image shape:", generated_test_images.shape)
```

Input noise shape: (4, 100)

Generated image shape: (4, 28, 28, 1)

```
In [17]: # =====  
# Visualize images generated before training  
# =====  
  
plt.figure(figsize=(8, 2))  
  
for i in range(4):  
    plt.subplot(1, 4, i + 1)  
  
    # Convert generated image values from [-1, 1] to [0, 1]  
    image = (generated_test_images[i, :, :, 0] + 1) / 2  
  
    plt.imshow(image, cmap="gray")  
    plt.axis("off")  
    plt.title("Untrained")  
  
plt.tight_layout()  
plt.show()
```



Observation

The images generated before training do not contain recognizable fashion patterns because the Generator has not yet learned the distribution of the Fashion-MNIST dataset.

These initial outputs provide a baseline for comparing the quality of images after adversarial training.

16. Generator Architecture Analysis

The Generator transforms a 100-dimensional random noise vector into a $28 \times 28 \times 1$ grayscale image.

The architecture progressively increases the spatial dimensions:

- Dense layer: Projects the latent vector into a feature representation.
- Reshape layer: Converts the feature vector into a 7×7 feature map.
- First Conv2DTranspose: Upsamples 7×7 to 14×14 .
- Second Conv2DTranspose: Upsamples 14×14 to 28×28 .
- Final Conv2DTranspose: Produces a single-channel grayscale image.
- Batch Normalization: Helps stabilize the training process.

- ReLU: Provides nonlinear transformation in intermediate layers.
- Tanh: Produces output values in the range $[-1, 1]$, matching the normalized training images.

In [18]: *# Check the output range of the untrained Generator*

```
print("Generated image minimum:", generated_test_images.numpy().min())
print("Generated image maximum:", generated_test_images.numpy().max())
```

Generated image minimum: -0.0352006

Generated image maximum: 0.10485543

17. Define Discriminator

The Discriminator is a convolutional neural network that receives a 28×28 grayscale image and predicts whether the image is a real sample from the Fashion-MNIST dataset or a synthetic image produced by the Generator.

The Discriminator uses convolutional layers to extract spatial features from the images.

Unlike the Generator, the Discriminator reduces the spatial dimensions while increasing the number of learned feature channels.

The final output is a single value representing the probability that the input image is real.

In [19]: *# =====*
Section 17: Build the DCGAN Discriminator
=====

```
def build_discriminator():
    """
    Creates the Discriminator network.

    Input:
        28 x 28 x 1 grayscale image

    Output:
        Single probability indicating real or fake.
    """

    model = keras.Sequential(name="DCGAN_Discriminator")

    # Input image: 28 x 28 x 1
    model.add(
        layers.Input(shape=(28, 28, 1))
    )

    # Extract low-level visual features
    model.add(
        layers.Conv2D(
            64,
            kernel_size=5,
```

```

        strides=2,
        padding="same"
    )
)
model.add(
    layers.LeakyReLU(negative_slope=0.2)
)
model.add(
    layers.Dropout(0.3)
)

# Extract deeper visual features
model.add(
    layers.Conv2D(
        128,
        kernel_size=5,
        strides=2,
        padding="same"
    )
)
model.add(
    layers.LeakyReLU(negative_slope=0.2)
)
model.add(
    layers.Dropout(0.3)
)

# Convert feature maps into a single prediction
model.add(
    layers.Flatten()
)

model.add(
    layers.Dense(1)
)

return model

# Create the Discriminator
discriminator = build_discriminator()

print("Discriminator created successfully.")

```

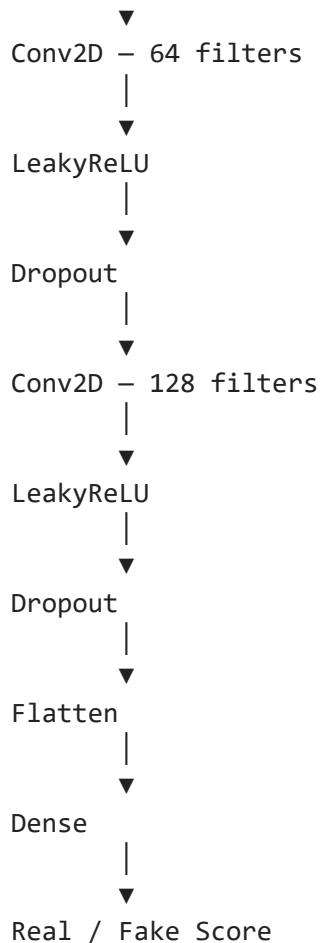
Discriminator created successfully.

18. Discriminator Architecture

The Discriminator receives a $28 \times 28 \times 1$ image and progressively extracts visual features using convolutional layers.

The architecture follows this general structure:

28 × 28 × 1 Image
|



18.1 Discriminator Model Summary and Test

The following cells display the Discriminator architecture and test its output on both real and generated images.

The Discriminator produces a score for each input image, which is used later during GAN training to distinguish real images from generated images.

```

In [20]: # =====
# Test the Discriminator with real and generated images
# =====

# Select four real images from the training data
real_test_images = X_train[:4]

# Generate four fake images using the Generator
fake_test_images = generator(test_noise, training=False)

# Obtain discriminator scores
real_scores = discriminator(real_test_images, training=False)
fake_scores = discriminator(fake_test_images, training=False)

print("Real image batch shape:", real_test_images.shape)
print("Fake image batch shape:", fake_test_images.shape)

```

```
print("\nDiscriminator output shape:", real_scores.shape)
```

Real image batch shape: (4, 28, 28, 1)

Fake image batch shape: (4, 28, 28, 1)

Discriminator output shape: (4, 1)

19. Model Summaries

The Generator and Discriminator architectures are displayed to verify their structures before beginning adversarial training.

The Generator converts random latent vectors into 28×28 grayscale images, while the Discriminator converts an image into a real/fake prediction score.

In [21]: *# Display both model summaries*

```
print("=" * 70)
print("GENERATOR SUMMARY")
print("=" * 70)

generator.summary()

print("\n" + "=" * 70)
print("DISCRIMINATOR SUMMARY")
print("=" * 70)

discriminator.summary()
```

```
=====
GENERATOR SUMMARY
=====
```

Model: "DCGAN_Generator"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 6272)	627,200
batch_normalization (BatchNormalization)	(None, 6272)	25,088
re_lu (ReLU)	(None, 6272)	0
reshape (Reshape)	(None, 7, 7, 128)	0
conv2d_transpose (Conv2DTranspose)	(None, 14, 14, 64)	204,800
batch_normalization_1 (BatchNormalization)	(None, 14, 14, 64)	256
re_lu_1 (ReLU)	(None, 14, 14, 64)	0
conv2d_transpose_1 (Conv2DTranspose)	(None, 28, 28, 32)	51,200
batch_normalization_2 (BatchNormalization)	(None, 28, 28, 32)	128
re_lu_2 (ReLU)	(None, 28, 28, 32)	0
conv2d_transpose_2 (Conv2DTranspose)	(None, 28, 28, 1)	801

Total params: 909,473 (3.47 MB)

Trainable params: 896,737 (3.42 MB)

Non-trainable params: 12,736 (49.75 KB)

=====

DISCRIMINATOR SUMMARY

=====

Model: "DCGAN_Discriminator"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 14, 14, 64)	1,664
leaky_re_lu (LeakyReLU)	(None, 14, 14, 64)	0
dropout (Dropout)	(None, 14, 14, 64)	0
conv2d_1 (Conv2D)	(None, 7, 7, 128)	204,928
leaky_re_lu_1 (LeakyReLU)	(None, 7, 7, 128)	0
dropout_1 (Dropout)	(None, 7, 7, 128)	0
flatten (Flatten)	(None, 6272)	0
dense_1 (Dense)	(None, 1)	6,273

Total params: 212,865 (831.50 KB)

Trainable params: 212,865 (831.50 KB)

Non-trainable params: 0 (0.00 B)

20. Define Loss Functions

The DCGAN uses two separate loss functions:

Discriminator Loss

The Discriminator learns to classify:

- Real images as real.
- Generated images as fake.

Its loss is calculated using the predictions for both real and generated images.

Generator Loss

The Generator attempts to produce images that the Discriminator classifies as real.

Therefore, the Generator is trained using the Discriminator's prediction on generated images.

Binary Cross-Entropy is used to calculate both losses.

The adversarial training objective can be summarized as:

Generator:

Random Noise → Generated Image → Discriminator → Generator Loss

Discriminator:

Real Image → Discriminator → Real Loss

Fake Image → Discriminator → Fake Loss

↓

Discriminator Loss

```
In [22]: # =====
# Section 20: Define GAN Loss Functions
# =====

# Binary Cross-Entropy is commonly used for GAN adversarial Loss
cross_entropy = keras.losses.BinaryCrossentropy(from_logits=True)

def discriminator_loss(real_output, fake_output):
    """
    Calculate the total Discriminator loss.

    Real images should be classified as 1.
    Generated images should be classified as 0.
    """

    real_loss = cross_entropy(
        tf.ones_like(real_output),
        real_output
    )

    fake_loss = cross_entropy(
        tf.zeros_like(fake_output),
        fake_output
    )

    total_loss = real_loss + fake_loss

    return total_loss

def generator_loss(fake_output):
    """
    Calculate the Generator loss.

    The Generator wants the Discriminator to classify
    generated images as real (label = 1).
    """

    loss = cross_entropy(
        tf.ones_like(fake_output),
        fake_output
    )

    return loss

print("GAN loss functions defined successfully.")
```

GAN loss functions defined successfully.

21. Define Optimizers

The Generator and Discriminator use separate Adam optimizers.

Adam is selected because it provides adaptive learning rates and is commonly used for training neural networks.

Separate optimizers are required because the Generator and Discriminator have different trainable parameters and different optimization objectives.

```
In [23]: # =====  
# Section 21: Define Optimizers  
# =====  
  
# Learning rate used for both networks  
LEARNING_RATE = 0.0002  
  
# Adam optimizer for the Generator  
generator_optimizer = keras.optimizers.Adam(  
    learning_rate=LEARNING_RATE,  
    beta_1=0.5  
)  
  
# Adam optimizer for the Discriminator  
discriminator_optimizer = keras.optimizers.Adam(  
    learning_rate=LEARNING_RATE,  
    beta_1=0.5  
)  
  
print("Optimizers created successfully.")  
print("Learning rate:", LEARNING_RATE)  
print("Adam beta_1:", 0.5)
```

Optimizers created successfully.

Learning rate: 0.0002

Adam beta_1: 0.5

Random Noise ↓ Generator ↓ Fake Images ↓ Discriminator ↓ Calculate Loss ↓ Calculate Gradients ↓ Update Generator + Discriminator

22. Define Training Step

A single training step performs one adversarial update of the Generator and Discriminator.

The process is:

1. Generate random noise.
2. Generate synthetic images using the Generator.
3. Pass real images through the Discriminator.

4. Pass generated images through the Discriminator.
5. Calculate Generator and Discriminator losses.
6. Calculate gradients using automatic differentiation.
7. Update the Generator parameters.
8. Update the Discriminator parameters.

The `tf.GradientTape` mechanism is used to calculate gradients for both networks.

```
In [24]: # =====
# Section 22: Define One GAN Training Step
# =====

@tf.function
def train_step(real_images):
    """
    Performs one adversarial training step.

    Parameters:
        real_images: A batch of real Fashion-MNIST images.

    Returns:
        Generator loss and Discriminator loss.
    """

    # Get the number of images in the current batch
    batch_size = tf.shape(real_images)[0]

    # Generate random Latent vectors
    noise = tf.random.normal(
        shape=(batch_size, LATENT_DIM)
    )

    # Record operations needed for gradient calculation
    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:

        # Generate fake images
        generated_images = generator(
            noise,
            training=True
        )

        # Discriminator predictions
        real_output = discriminator(
            real_images,
            training=True
        )

        fake_output = discriminator(
            generated_images,
            training=True
        )

        # Calculate both losses
        gen_loss = generator_loss(fake_output)
```

```

        disc_loss = discriminator_loss(
            real_output,
            fake_output
        )

        # Calculate gradients for the Generator
        gradients_of_generator = gen_tape.gradient(
            gen_loss,
            generator.trainable_variables
        )

        # Calculate gradients for the Discriminator
        gradients_of_discriminator = disc_tape.gradient(
            disc_loss,
            discriminator.trainable_variables
        )

        # Update Generator parameters
        generator_optimizer.apply_gradients(
            zip(
                gradients_of_generator,
                generator.trainable_variables
            )
        )

        # Update Discriminator parameters
        discriminator_optimizer.apply_gradients(
            zip(
                gradients_of_discriminator,
                discriminator.trainable_variables
            )
        )

        return gen_loss, disc_loss

print("Training step defined successfully.")

```

Training step defined successfully.

```

In [25]: # =====
# Test one complete GAN training step
# =====

# Take one batch of real images
real_batch = next(iter(train_dataset))

# Run one adversarial training step
gen_loss, disc_loss = train_step(real_batch)

print("One training step completed successfully.")
print("Generator loss:", float(gen_loss))
print("Discriminator loss:", float(disc_loss))

```

One training step completed successfully.

Generator loss: 0.6983590722084045

Discriminator loss: 1.366397500038147

23. Fixed Noise for Training Progress

A fixed set of random noise vectors is created to monitor the Generator throughout training.

Using the same latent vectors at different training stages allows us to observe how generated images evolve as the Generator learns.

```
In [26]: # =====
# Section 23: Fixed Noise for Monitoring Training Progress
# =====

# Number of images used for progress visualization
NUM_EXAMPLES = 16

# Create fixed latent vectors
fixed_noise = tf.random.normal(
    shape=(NUM_EXAMPLES, LATENT_DIM),
    seed=SEED
)

print("Fixed noise shape:", fixed_noise.shape)
```

Fixed noise shape: (16, 100)

```
In [27]: # Generate images from the fixed noise vectors

def generate_and_plot_images(model, noise, title="Generated Images"):
    """
    Generate and display images from a fixed set of latent vectors.
    """

    predictions = model(
        noise,
        training=False
    )

    plt.figure(figsize=(8, 8))

    for i in range(predictions.shape[0]):
        plt.subplot(4, 4, i + 1)

        # Convert pixel values from [-1, 1] to [0, 1]
        image = (predictions[i, :, :, 0] + 1) / 2

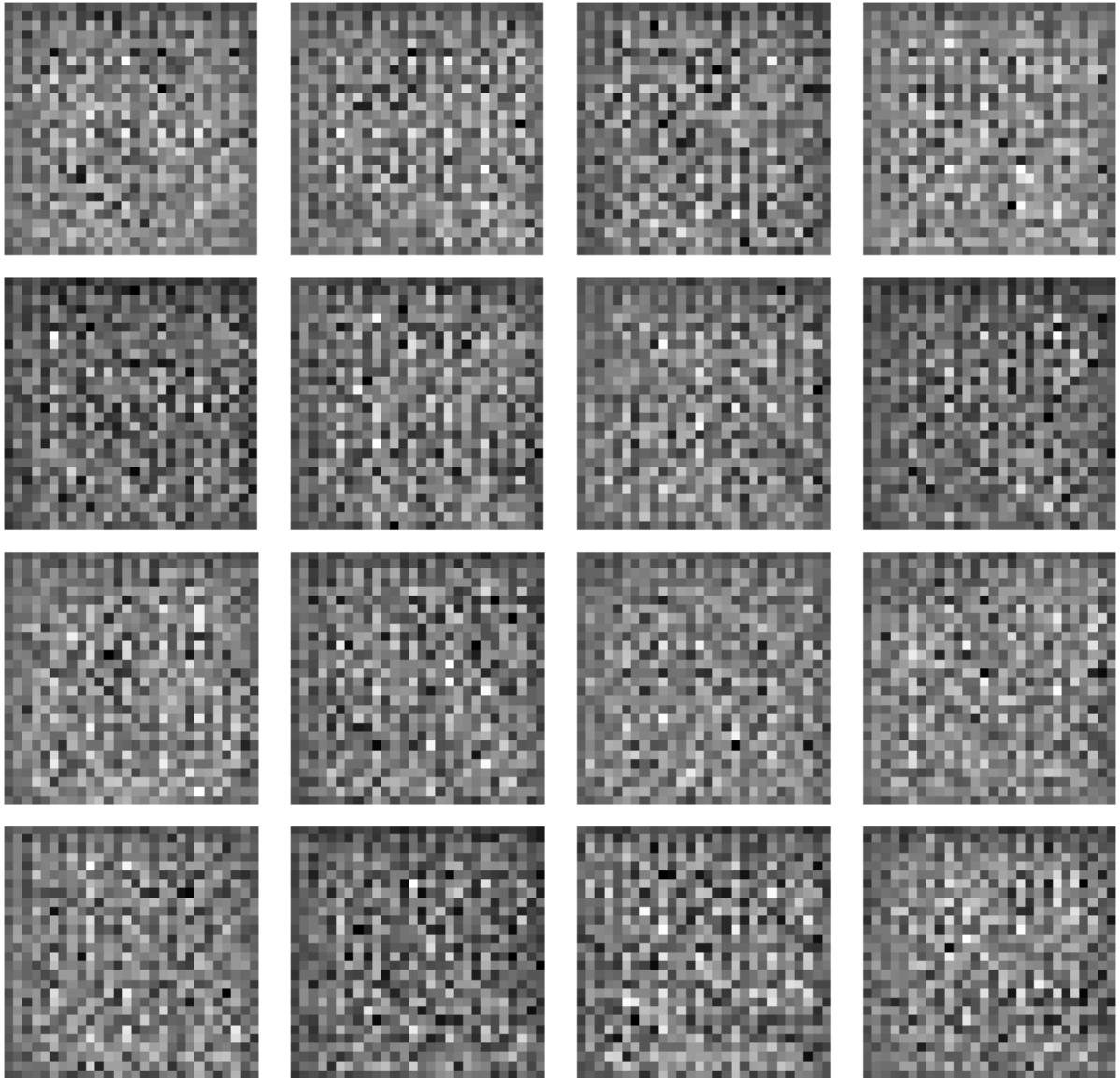
        plt.imshow(image, cmap="gray")
        plt.axis("off")

    plt.suptitle(title)
    plt.tight_layout()
    plt.show()
```

In [28]: *# Generate a reference image grid before full training*

```
generate_and_plot_images(  
    generator,  
    fixed_noise,  
    title="Generator Output Before Full Training"  
)
```

Generator Output Before Full Training



24. DCGAN Training

The DCGAN is trained using an adversarial training process.

For each batch:

1. A batch of real Fashion-MNIST images is selected.
2. Random noise vectors are generated.

3. The Generator creates synthetic images.
4. The Discriminator evaluates both real and synthetic images.
5. Generator and Discriminator losses are calculated.
6. Gradients are calculated using backpropagation.
7. Both networks are updated using their respective optimizers.

The process is repeated for multiple epochs so that the Generator gradually learns to generate images that resemble the Fashion-MNIST training distribution.

A fixed noise vector is used periodically to visualize the Generator's progress throughout training.

```
In [29]: # =====
# Section 24: DCGAN Training Configuration
# =====

# Number of epochs for the initial training experiment
EPOCHS = 20

# Number of examples saved for progress visualization
SAVE_INTERVAL = 5

# Lists used to store training losses
generator_losses = []
discriminator_losses = []

# Create folders for generated images and results
os.makedirs("generated_images", exist_ok=True)
os.makedirs("results", exist_ok=True)

print("Training configuration ready.")
print("Epochs:", EPOCHS)
print("Image save interval:", SAVE_INTERVAL)
```

Training configuration ready.

Epochs: 20

Image save interval: 5

```
In [30]: # =====
# Function to save generated images during training
# =====

def save_generated_images(model, noise, epoch):
    """
    Generate images using fixed noise and save them to disk.
    """

    predictions = model(
        noise,
        training=False
    )

    plt.figure(figsize=(8, 8))
```

```

for i in range(16):
    plt.subplot(4, 4, i + 1)

    # Convert [-1, 1] back to [0, 1]
    image = (predictions[i, :, :, 0] + 1) / 2

    plt.imshow(image, cmap="gray")
    plt.axis("off")

plt.suptitle(f"DCGAN Generated Images - Epoch {epoch}")

plt.tight_layout()

# Save the generated image grid
filename = f"generated_images/epoch_{epoch:03d}.png"
plt.savefig(filename, dpi=150, bbox_inches="tight")

plt.show()
plt.close()

print(f"Saved: {filename}")

```

```

In [31]: # =====
# DCGAN Training Loop
# =====

print("Starting DCGAN training...\n")

for epoch in range(1, EPOCHS + 1):

    # Store losses for the current epoch
    epoch_generator_loss = []
    epoch_discriminator_loss = []

    # Train using every batch in the dataset
    for real_images in train_dataset:

        # Perform one adversarial training step
        gen_loss, disc_loss = train_step(real_images)

        # Store the loss values
        epoch_generator_loss.append(float(gen_loss))
        epoch_discriminator_loss.append(float(disc_loss))

    # Calculate average loss for the epoch
    avg_gen_loss = np.mean(epoch_generator_loss)
    avg_disc_loss = np.mean(epoch_discriminator_loss)

    # Store the average losses
    generator_losses.append(avg_gen_loss)
    discriminator_losses.append(avg_disc_loss)

    # Display epoch results
    print(
        f"Epoch {epoch:03d}/{EPOCHS} | "
        f"Generator Loss: {avg_gen_loss:.4f} | "

```

```
f"Discriminator Loss: {avg_disc_loss:.4f}"
)

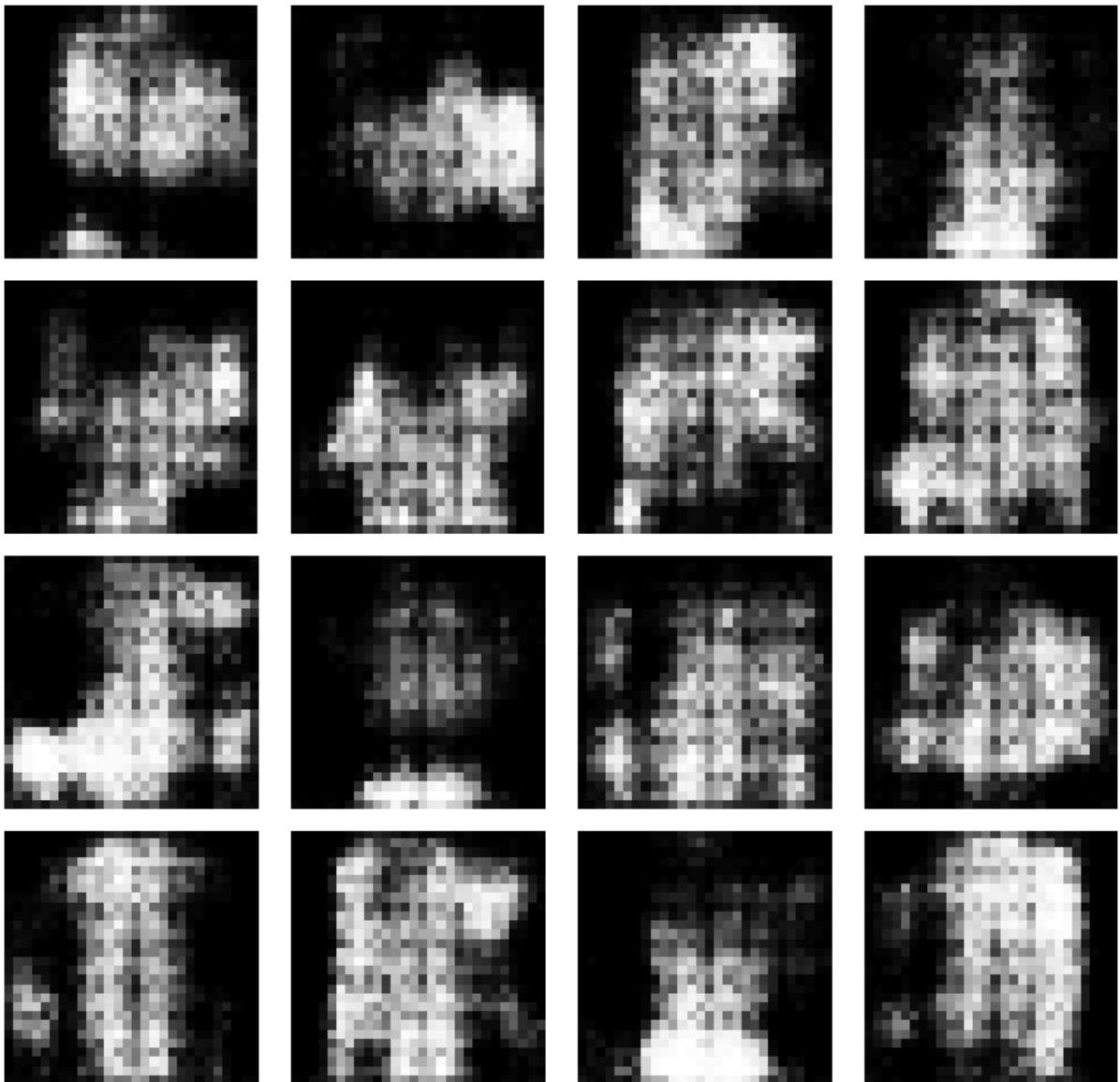
# Save generated images at selected intervals
if epoch == 1 or epoch % SAVE_INTERVAL == 0:
    save_generated_images(
        generator,
        fixed_noise,
        epoch
    )

print("\nDCGAN training completed successfully.")
```

Starting DCGAN training...

Epoch 001/20 | Generator Loss: 0.7594 | Discriminator Loss: 1.2889

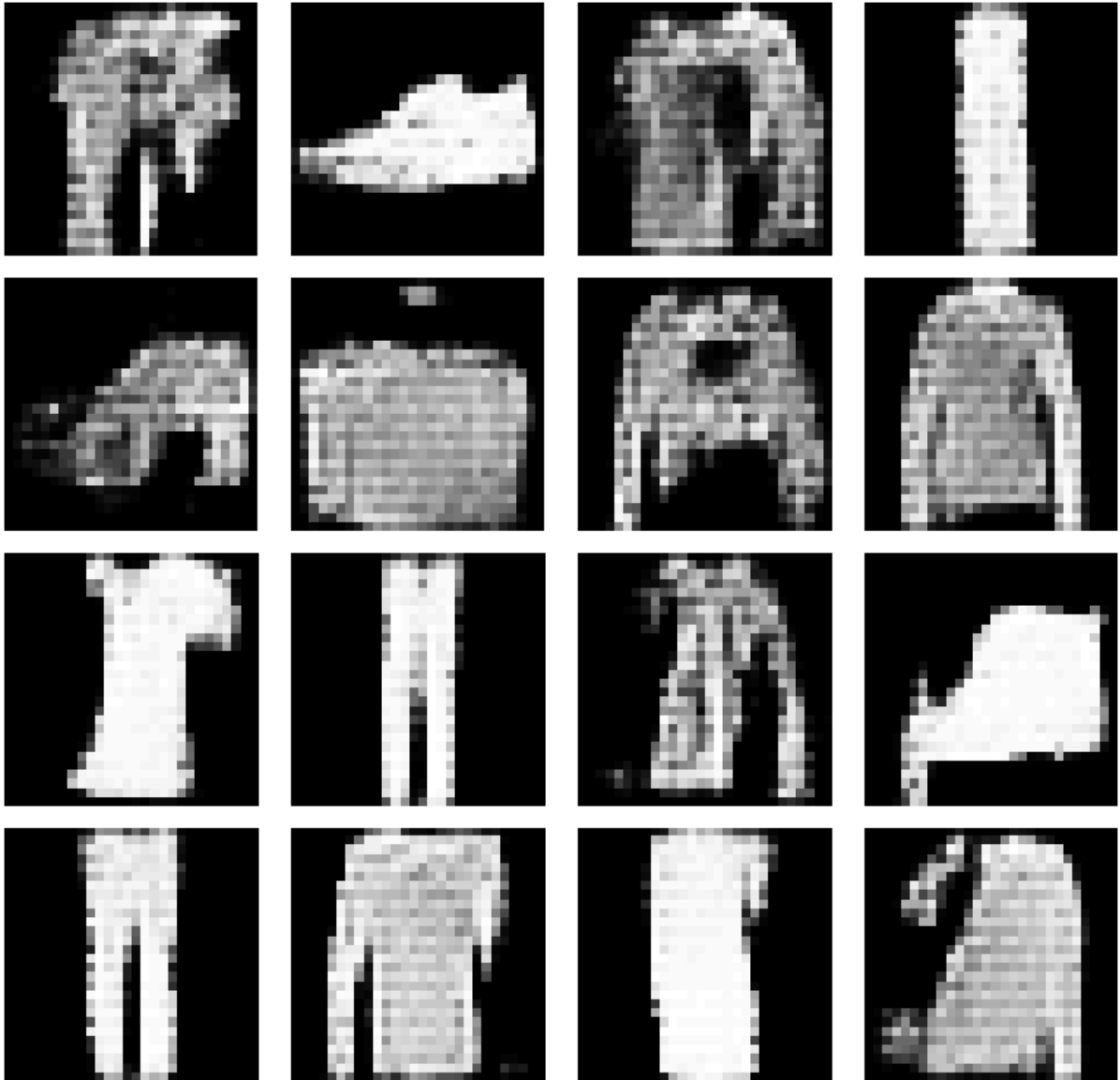
DCGAN Generated Images - Epoch 1



Saved: generated_images/epoch_001.png

Epoch 002/20	Generator Loss: 0.7688	Discriminator Loss: 1.2869
Epoch 003/20	Generator Loss: 0.8844	Discriminator Loss: 1.2199
Epoch 004/20	Generator Loss: 0.7953	Discriminator Loss: 1.2849
Epoch 005/20	Generator Loss: 0.7673	Discriminator Loss: 1.3192

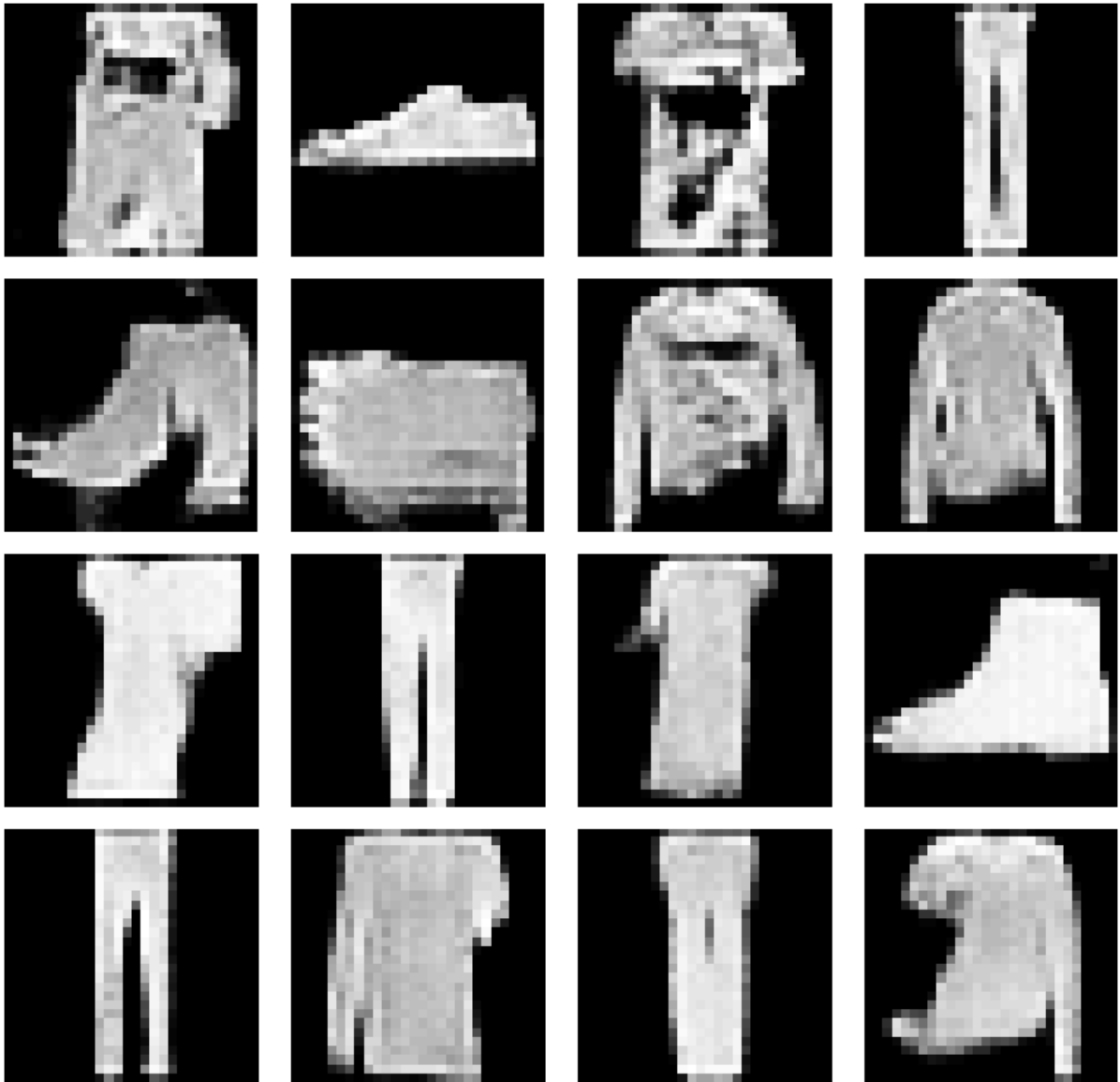
DCGAN Generated Images - Epoch 5



Saved: generated_images/epoch_005.png

Epoch 006/20	Generator Loss: 0.7590	Discriminator Loss: 1.3266
Epoch 007/20	Generator Loss: 0.7602	Discriminator Loss: 1.3280
Epoch 008/20	Generator Loss: 0.7543	Discriminator Loss: 1.3337
Epoch 009/20	Generator Loss: 0.7534	Discriminator Loss: 1.3362
Epoch 010/20	Generator Loss: 0.7562	Discriminator Loss: 1.3346

DCGAN Generated Images - Epoch 10



Saved: generated_images/epoch_010.png

Epoch 011/20 | Generator Loss: 0.7560 | Discriminator Loss: 1.3335

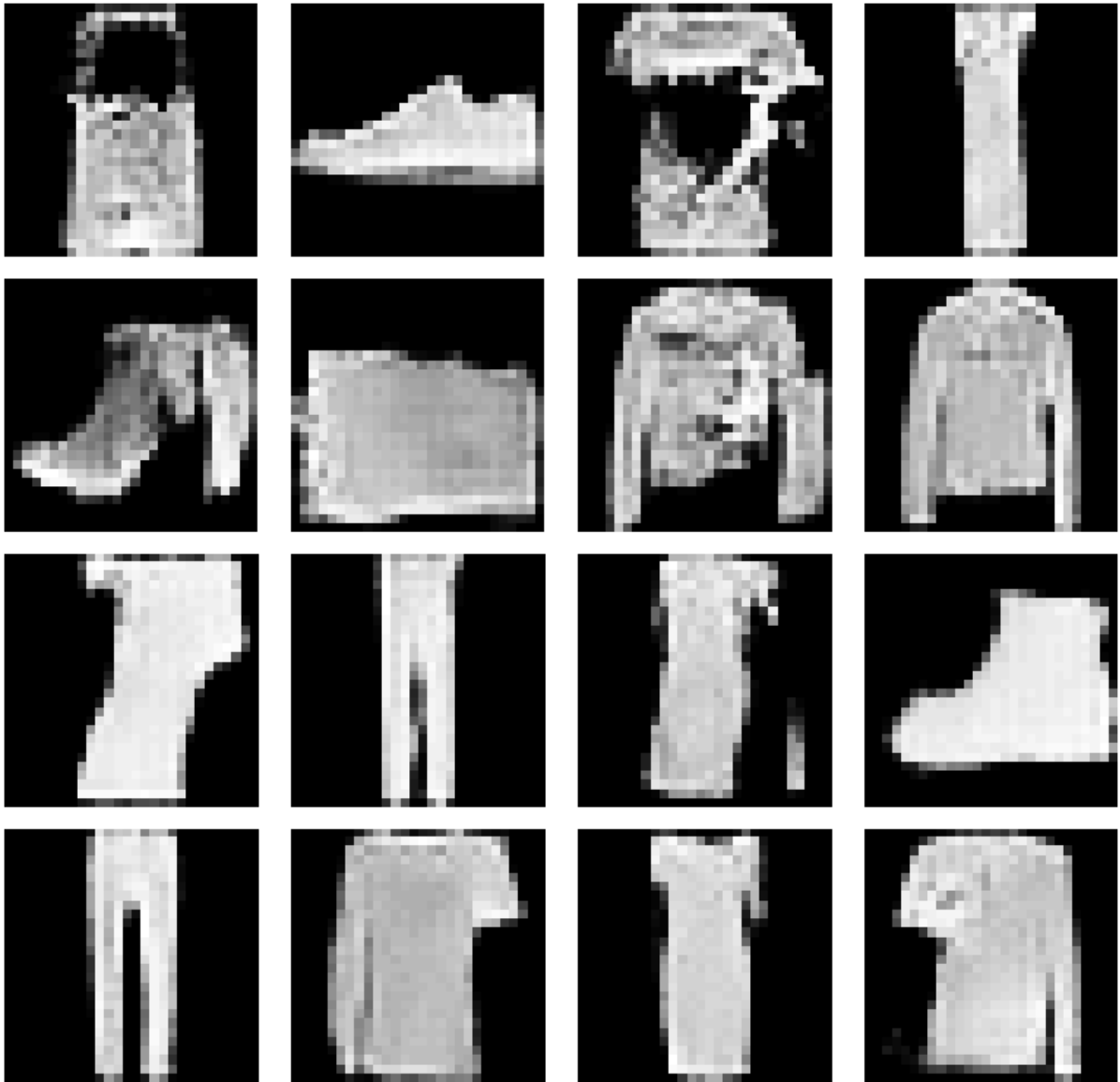
Epoch 012/20 | Generator Loss: 0.7569 | Discriminator Loss: 1.3334

Epoch 013/20 | Generator Loss: 0.7546 | Discriminator Loss: 1.3371

Epoch 014/20 | Generator Loss: 0.7548 | Discriminator Loss: 1.3388

Epoch 015/20 | Generator Loss: 0.7518 | Discriminator Loss: 1.3428

DCGAN Generated Images - Epoch 15



Saved: generated_images/epoch_015.png

Epoch 016/20 | Generator Loss: 0.7504 | Discriminator Loss: 1.3443

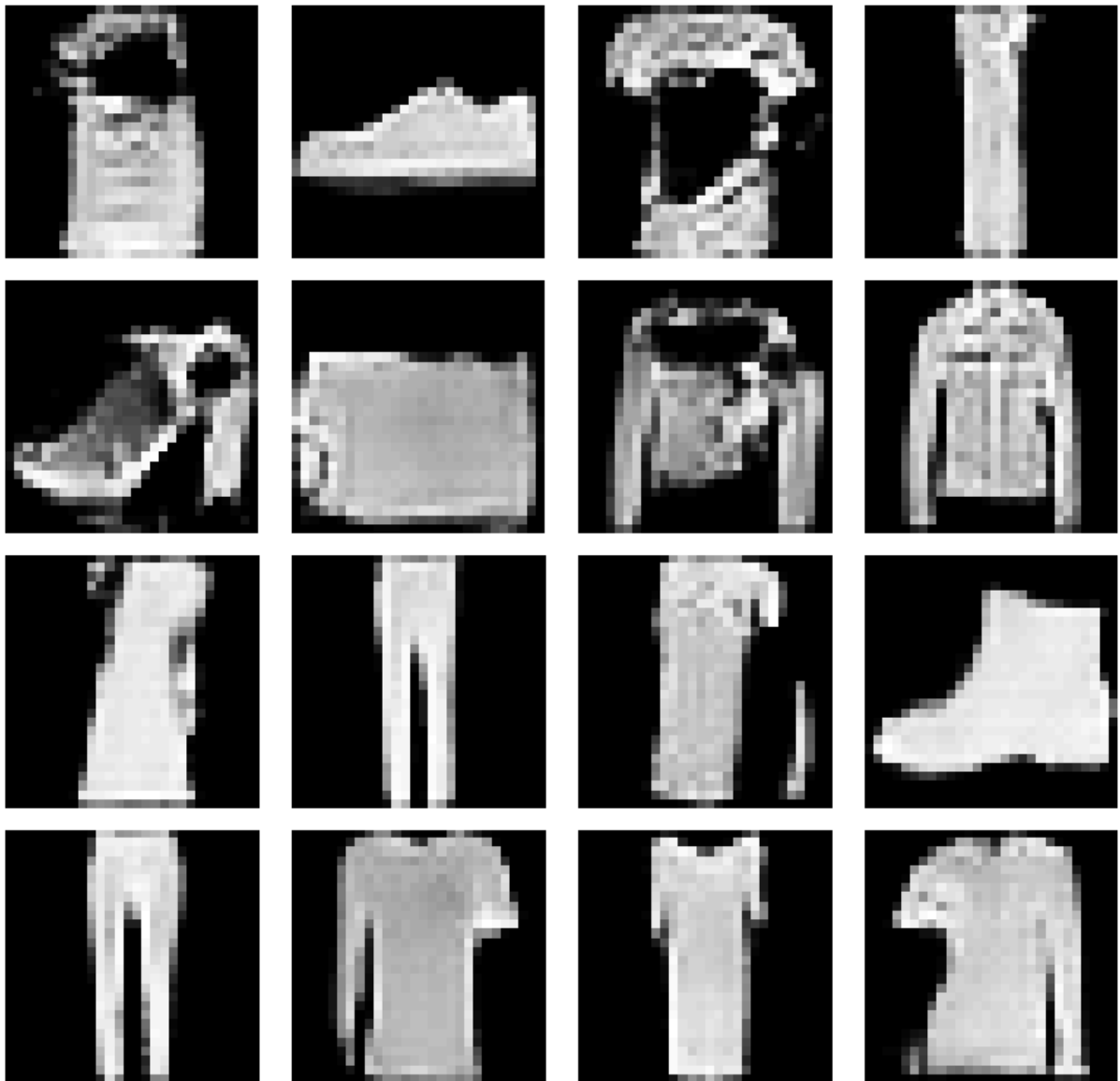
Epoch 017/20 | Generator Loss: 0.7508 | Discriminator Loss: 1.3442

Epoch 018/20 | Generator Loss: 0.7494 | Discriminator Loss: 1.3446

Epoch 019/20 | Generator Loss: 0.7472 | Discriminator Loss: 1.3477

Epoch 020/20 | Generator Loss: 0.7478 | Discriminator Loss: 1.3476

DCGAN Generated Images - Epoch 20



Saved: generated_images/epoch_020.png

DCGAN training completed successfully.

25. Save the Trained Models

After completing the initial training, the trained Generator and Discriminator are saved so that the learned model parameters can be reused later without retraining the complete GAN.

The Generator can be used independently to generate new synthetic Fashion-MNIST-style images.

```
In [32]: # =====
# Section 25: Save Trained Models
# =====

# Create model directories if they do not already exist
```

```

os.makedirs("models/generator", exist_ok=True)
os.makedirs("models/discriminator", exist_ok=True)

# Save the trained Generator
generator.save("models/generator/dcgan_generator.keras")

# Save the trained Discriminator
discriminator.save("models/discriminator/dcgan_discriminator.keras")

print("Trained Generator saved successfully.")
print("Trained Discriminator saved successfully.")

```

Trained Generator saved successfully.
Trained Discriminator saved successfully.

26. Generate Final Synthetic Images

The trained Generator is used to create a new set of synthetic Fashion-MNIST images.

These images were not directly copied from the training dataset. Instead, they are generated from random latent vectors learned through the adversarial training process.

```

In [33]: # =====
# Section 26: Generate Final Synthetic Images
# =====

# Generate random latent vectors
final_noise = tf.random.normal(
    shape=(25, LATENT_DIM),
    seed=SEED + 1
)

# Generate synthetic images
final_generated_images = generator(
    final_noise,
    training=False
)

print("Generated image batch shape:", final_generated_images.shape)

```

Generated image batch shape: (25, 28, 28, 1)

```

In [34]: # Display the final generated image grid

plt.figure(figsize=(10, 10))

for i in range(25):
    plt.subplot(5, 5, i + 1)

    # Convert image values from [-1, 1] to [0, 1]
    image = (final_generated_images[i, :, :, 0] + 1) / 2

    plt.imshow(image, cmap="gray")
    plt.axis("off")

```

```
plt.suptitle("Final DCGAN Generated Fashion Images", fontsize=16)
plt.tight_layout()
plt.show()
```



27. Generator Loss Analysis

The Generator loss represents how effectively the Generator is attempting to produce images that the Discriminator classifies as real.

The Generator loss is recorded after each training epoch.

Plotting this loss allows us to observe how the Generator's optimization behavior changes during training.

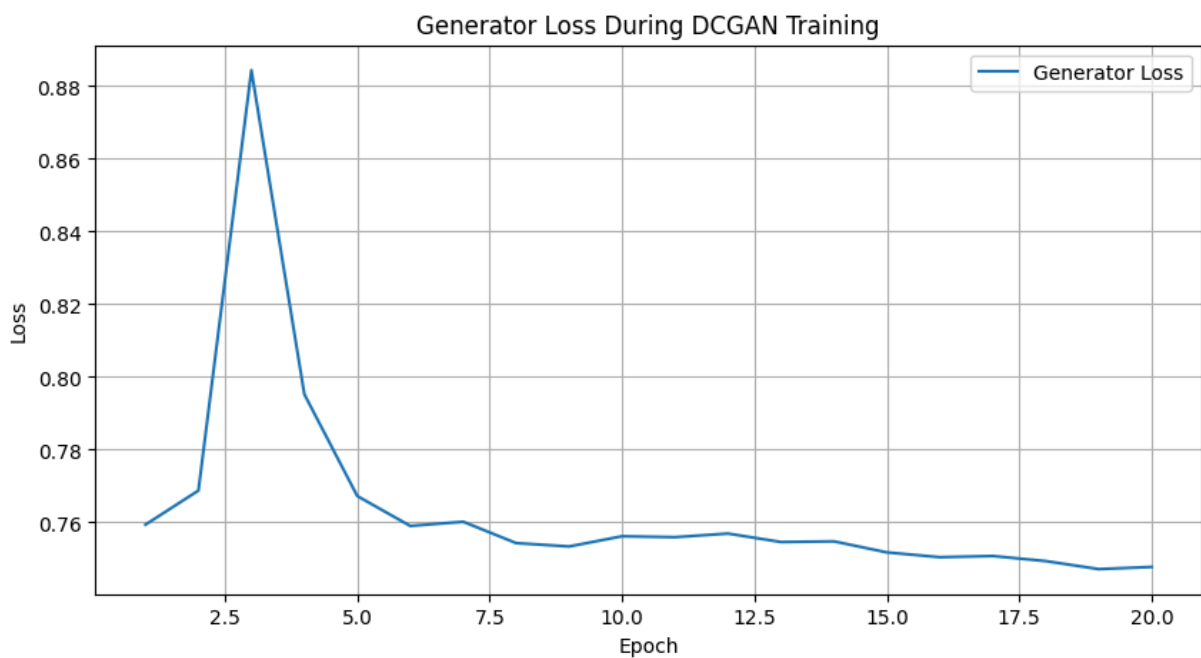
```
In [35]: # =====
# Section 27: Generator Loss Analysis
# =====
```

```
plt.figure(figsize=(10, 5))

plt.plot(
    range(1, len(generator_losses) + 1),
    generator_losses,
    label="Generator Loss"
)

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Generator Loss During DCGAN Training")
plt.legend()
plt.grid(True)

plt.show()
```



28. Discriminator Loss Analysis

The Discriminator loss measures the ability of the Discriminator to distinguish real Fashion-MNIST images from images generated by the Generator.

The loss is calculated using both real and generated samples.

The Discriminator loss is analyzed together with the Generator loss because GAN training involves competition between the two networks.

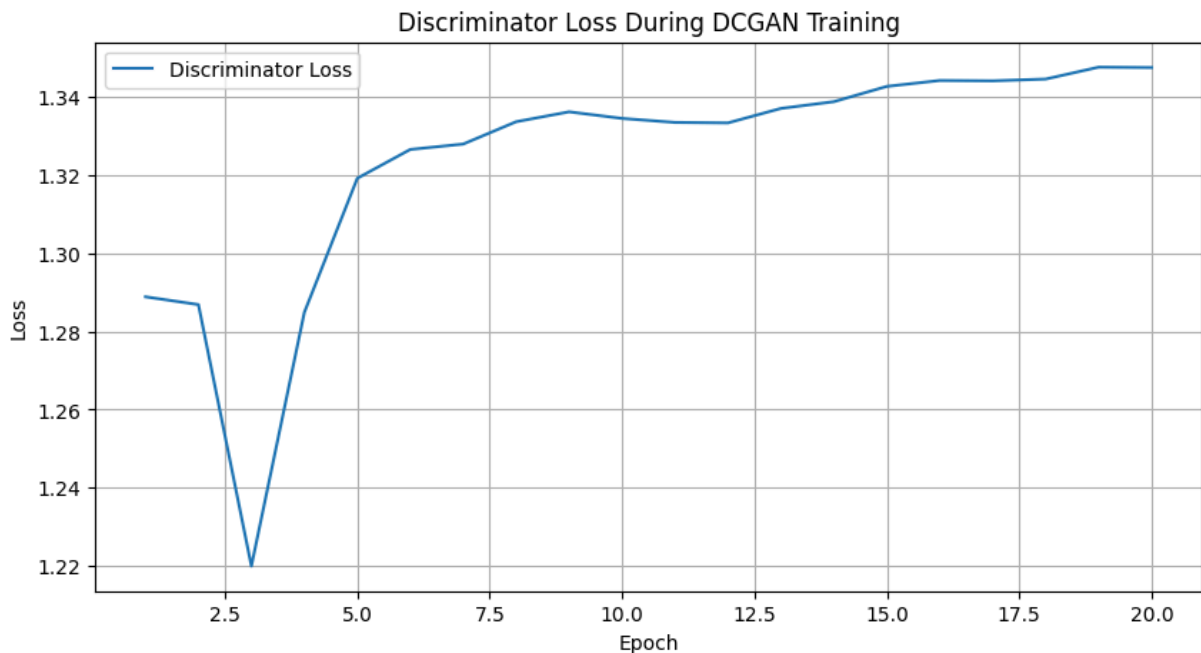
```
In [36]: # =====
# Section 28: Discriminator Loss Analysis
# =====

plt.figure(figsize=(10, 5))
```

```
plt.plot(
    range(1, len(discriminator_losses) + 1),
    discriminator_losses,
    label="Discriminator Loss"
)

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Discriminator Loss During DCGAN Training")
plt.legend()
plt.grid(True)

plt.show()
```



29. Combined Generator and Discriminator Loss Analysis

The Generator and Discriminator losses are plotted together to observe their adversarial training behavior.

GAN losses should not be interpreted in the same way as conventional classification accuracy. The two networks are competing against each other, so their losses can fluctuate as the adversarial training progresses.

Therefore, the loss curves are analyzed together with the generated images rather than using loss values alone to determine image quality.

```
In [37]: # =====
# Section 29: Combined Loss Analysis
# =====

plt.figure(figsize=(10, 5))
```

```

epochs_range = range(1, len(generator_losses) + 1)

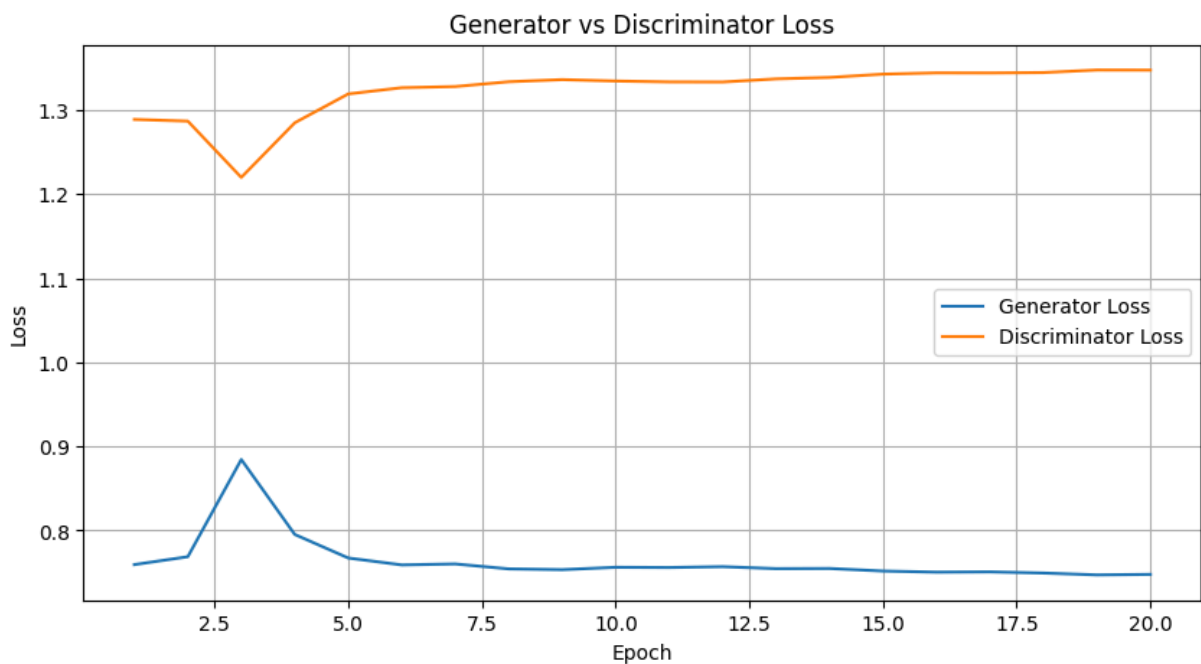
plt.plot(
    epochs_range,
    generator_losses,
    label="Generator Loss"
)

plt.plot(
    epochs_range,
    discriminator_losses,
    label="Discriminator Loss"
)

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Generator vs Discriminator Loss")
plt.legend()
plt.grid(True)

plt.show()

```



30. Save Training Loss Graphs

The Generator, Discriminator, and combined loss graphs are saved to the `results` folder for later analysis and documentation.

```

In [38]: # =====
# Save training loss graphs
# =====

# Save Generator Loss

```

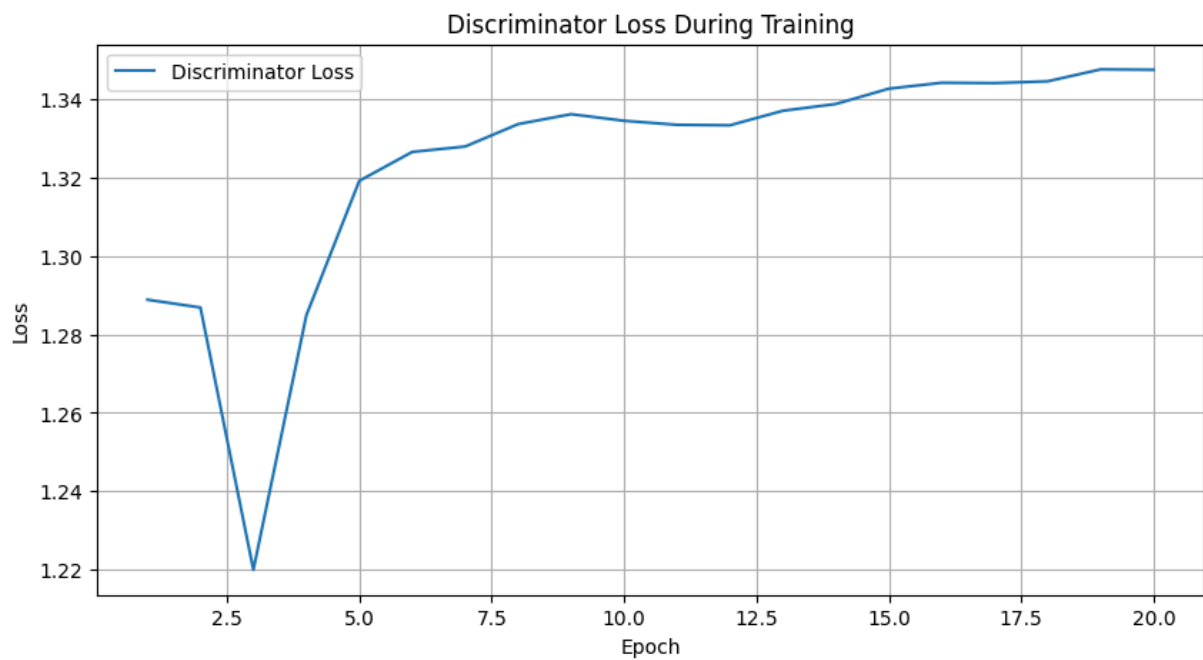
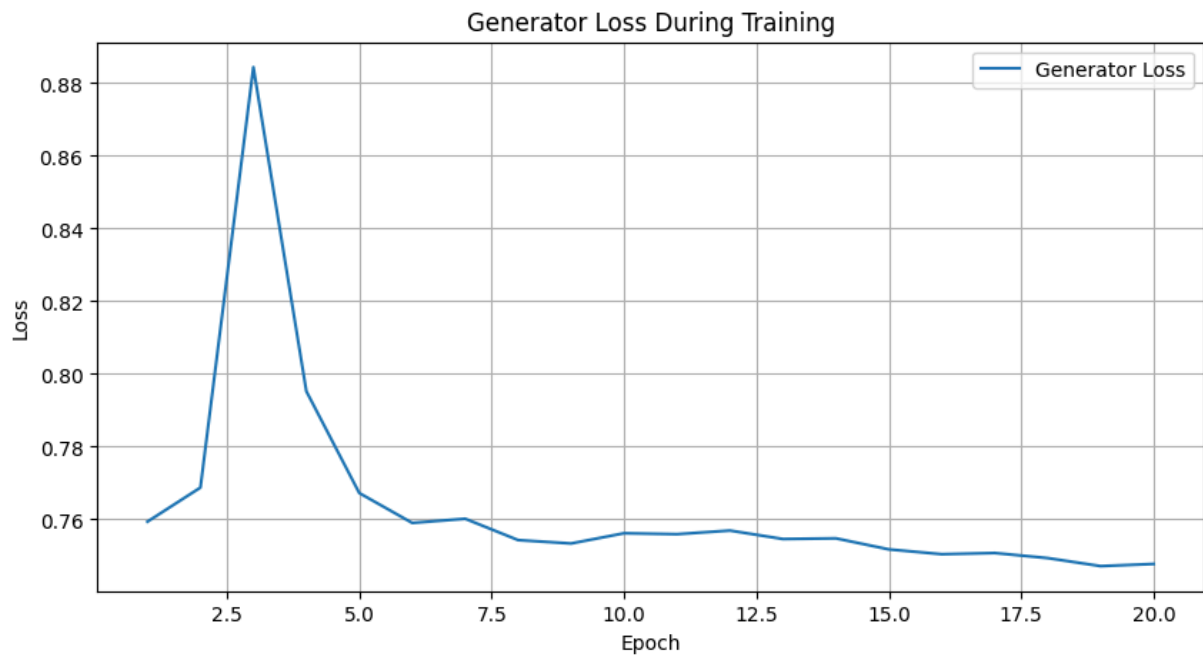
```
plt.figure(figsize=(10, 5))
plt.plot(
    epochs_range,
    generator_losses,
    label="Generator Loss"
)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Generator Loss During Training")
plt.legend()
plt.grid(True)
plt.savefig(
    "results/generator_loss.png",
    dpi=150,
    bbox_inches="tight"
)
plt.show()
plt.close()

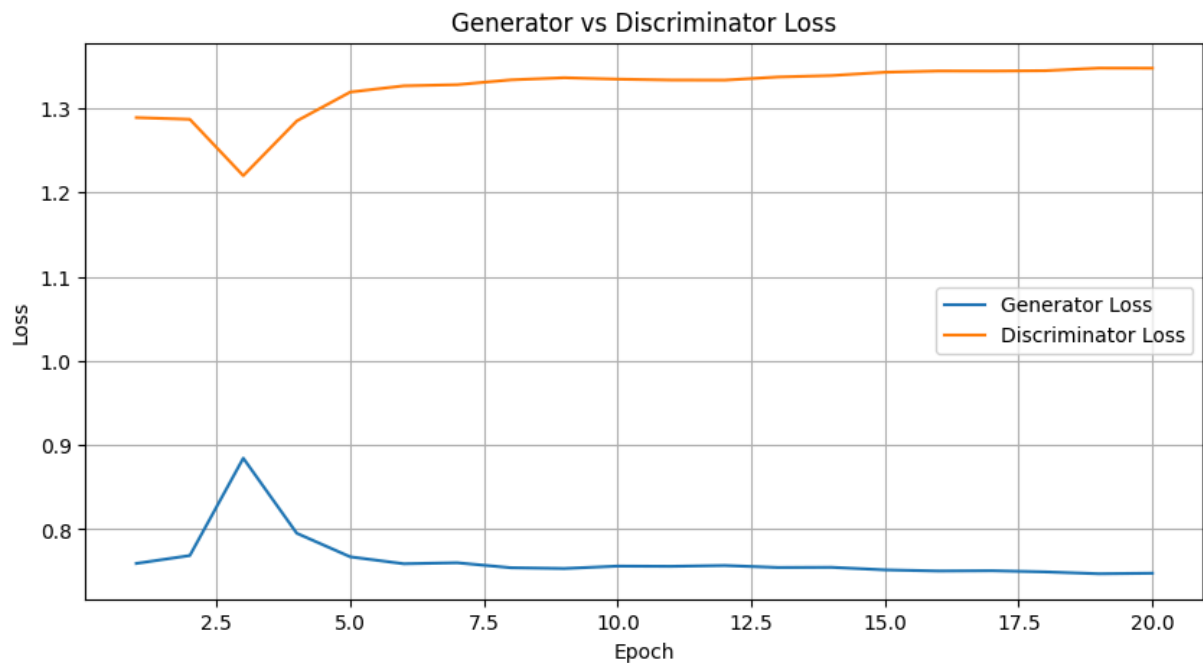
# Save Discriminator Loss
plt.figure(figsize=(10, 5))
plt.plot(
    epochs_range,
    discriminator_losses,
    label="Discriminator Loss"
)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Discriminator Loss During Training")
plt.legend()
plt.grid(True)
plt.savefig(
    "results/discriminator_loss.png",
    dpi=150,
    bbox_inches="tight"
)
plt.show()
plt.close()

# Save combined Loss
plt.figure(figsize=(10, 5))
plt.plot(
    epochs_range,
    generator_losses,
    label="Generator Loss"
)
plt.plot(
    epochs_range,
    discriminator_losses,
    label="Discriminator Loss"
)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Generator vs Discriminator Loss")
```

```
plt.legend()
plt.grid(True)
plt.savefig(
    "results/combined_loss.png",
    dpi=150,
    bbox_inches="tight"
)
plt.show()
plt.close()

print("Loss graphs saved successfully in the results folder.")
```





Loss graphs saved successfully in the results folder.

31. Real vs Generated Image Comparison

The quality of the trained Generator is visually examined by comparing real Fashion-MNIST images with synthetic images generated by the DCGAN.

Real images are taken directly from the dataset, while generated images are produced from previously unseen random latent vectors.

This comparison provides a qualitative evaluation of whether the Generator has learned meaningful visual patterns from the training data.

```
In [39]: # =====
# Section 31: Real vs Generated Image Comparison
# =====

# Select 10 real images
real_images = X_train[:10]

# Generate 10 synthetic images
comparison_noise = tf.random.normal(
    shape=(10, LATENT_DIM),
    seed=SEED + 2
)

generated_images = generator(
    comparison_noise,
    training=False
)

plt.figure(figsize=(12, 5))
```

```

# -----
# Real images
# -----
for i in range(10):
    plt.subplot(2, 10, i + 1)

    plt.imshow(
        real_images[i, :, :, 0],
        cmap="gray"
    )

    plt.axis("off")

    if i == 0:
        plt.ylabel("Real", fontsize=12)

# -----
# Generated images
# -----
for i in range(10):
    plt.subplot(2, 10, 10 + i + 1)

    # Convert [-1, 1] to [0, 1]
    image = (generated_images[i, :, :, 0] + 1) / 2

    plt.imshow(
        image,
        cmap="gray"
    )

    plt.axis("off")

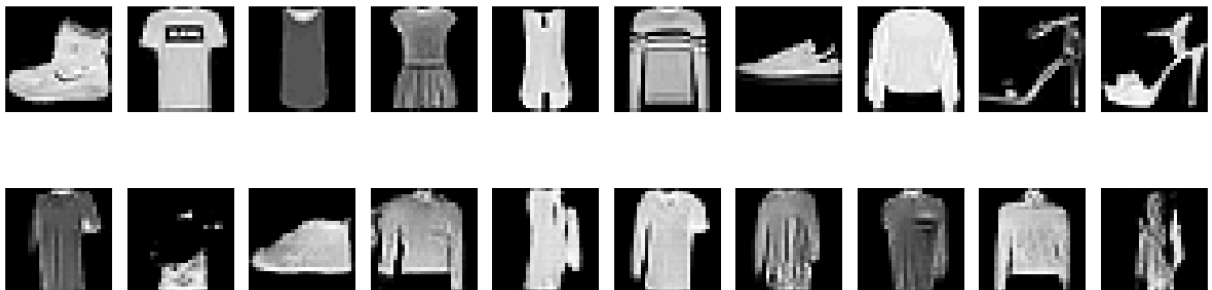
    if i == 0:
        plt.ylabel("Generated", fontsize=12)

plt.suptitle(
    "Real vs DCGAN Generated Images",
    fontsize=15
)

plt.tight_layout()
plt.show()

```

Real vs DCGAN Generated Images

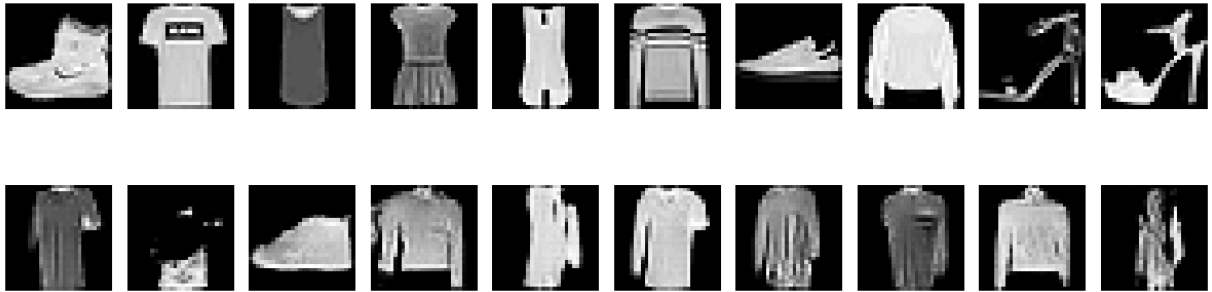


32. Save Real vs Generated Comparison

The real-versus-generated comparison is saved as an image so that it can be included in the final project results and presentation.

```
In [40]: # =====  
# Save Real vs Generated Comparison  
# =====  
  
plt.figure(figsize=(12, 5))  
  
for i in range(10):  
    plt.subplot(2, 10, i + 1)  
  
    plt.imshow(  
        real_images[i, :, :, 0],  
        cmap="gray"  
    )  
  
    plt.axis("off")  
  
for i in range(10):  
    plt.subplot(2, 10, 10 + i + 1)  
  
    image = (generated_images[i, :, :, 0] + 1) / 2  
  
    plt.imshow(  
        image,  
        cmap="gray"  
    )  
  
    plt.axis("off")  
  
plt.suptitle(  
    "Real vs DCGAN Generated Images",  
    fontsize=15  
)  
  
plt.tight_layout()  
  
plt.savefig(  
    "results/real_vs_generated.png",  
    dpi=150,  
    bbox_inches="tight"  
)  
  
plt.show()  
plt.close()  
  
print("Real vs generated comparison saved successfully.")
```

Real vs DCGAN Generated Images



Real vs generated comparison saved successfully.

33. Generated Image Pixel Statistics

Basic pixel statistics are calculated for the generated images.

The generated images are converted from the Generator's output range $[-1, 1]$ to the standard image range $[0, 1]$.

Mean, standard deviation, minimum, and maximum values are examined as basic numerical characteristics of the generated samples.

These statistics provide supporting information but are not sufficient by themselves to determine image quality.

```
In [41]: # =====
# Section 33: Generated Image Statistics
# =====

# Convert generated images from [-1, 1] to [0, 1]
generated_images_01 = (
    generated_images + 1
) / 2

generated_array = generated_images_01.numpy()

print("Generated Image Statistics")
print("=" * 45)

print(
    "Mean pixel value:",
    generated_array.mean()
)

print(
    "Standard deviation:",
    generated_array.std()
)

print(
```

```

    "Minimum pixel value:",
    generated_array.min()
)

print(
    "Maximum pixel value:",
    generated_array.max()
)

```

Generated Image Statistics

=====

Mean pixel value: 0.26287097

Standard deviation: 0.3299345

Minimum pixel value: 0.0

Maximum pixel value: 0.99787927

34. Advanced Evaluation Preparation

Advanced evaluation attempts to measure generated image quality using feature representations rather than relying only on visual inspection or GAN training losses.

Because Fashion-MNIST contains 28×28 grayscale images, this project uses a domain-specific feature extractor trained on Fashion-MNIST.

The feature extractor learns a 128-dimensional representation of the images. These feature representations are later used to compare real and generated images quantitatively.

The evaluation results are interpreted together with visual inspection and training-loss curves.

35. Create Evaluation Dataset

A separate evaluation set is prepared to compare the statistical characteristics of real Fashion-MNIST images and generated images.

The evaluation uses equal numbers of real and generated samples.

The real images represent the reference data distribution, while the generated images represent the distribution learned by the DCGAN.

```

In [42]: # =====
# Section 35: Create Evaluation Dataset
# =====

# Number of images used for quantitative evaluation
EVAL_SAMPLES = 10000

# Use a fixed number of real Fashion-MNIST images
real_eval_images = X_train[:EVAL_SAMPLES]

```

```

# Generate the same number of synthetic images
evaluation_noise = tf.random.normal(
    shape=(EVAL_SAMPLES, LATENT_DIM),
    seed=SEED + 10
)

generated_eval_images = generator(
    evaluation_noise,
    training=False
)

print("Real evaluation images:", real_eval_images.shape)
print("Generated evaluation images:", generated_eval_images.shape)

```

Real evaluation images: (10000, 28, 28, 1)
 Generated evaluation images: (10000, 28, 28, 1)

36. Build Fashion-MNIST Feature Extractor

A CNN-based feature extractor is trained on Fashion-MNIST classification.

The intermediate feature representation learned by this network is used for quantitative comparison between real and generated images.

This provides a domain-specific feature space because the extractor has learned visual characteristics directly from Fashion-MNIST.

The feature extractor is not part of the GAN itself. It is used only for post-training evaluation.

```

In [43]: # =====
# Section 36: Build Feature Extractor
# =====

def build_feature_extractor():
    """
    CNN used to learn Fashion-MNIST feature representations.
    """

    inputs = keras.Input(shape=(28, 28, 1))

    # First convolutional block
    x = layers.Conv2D(
        32,
        kernel_size=3,
        activation="relu",
        padding="same"
    )(inputs)

    x = layers.MaxPooling2D()(x)

    # Second convolutional block
    x = layers.Conv2D(
        64,
        kernel_size=3,

```

```

        activation="relu",
        padding="same"
    )(x)

    x = layers.MaxPooling2D()(x)

    # Third convolutional block
    x = layers.Conv2D(
        128,
        kernel_size=3,
        activation="relu",
        padding="same"
    )(x)

    x = layers.GlobalAveragePooling2D()(x)

    # Compact feature representation
    features = layers.Dense(
        128,
        activation="relu",
        name="feature_vector"
    )(x)

    outputs = layers.Dense(
        10,
        activation="softmax"
    )(features)

    model = keras.Model(
        inputs,
        outputs,
        name="FashionMNIST_Feature_Extractor"
    )

    feature_model = keras.Model(
        inputs,
        features,
        name="FashionMNIST_Feature_Model"
    )

    return model, feature_model

feature_classifier, feature_extractor = build_feature_extractor()

feature_classifier.summary()

```

Model: "FashionMNIST_Feature_Extractor"

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 28, 28, 1)	0
conv2d_2 (Conv2D)	(None, 28, 28, 32)	320
max_pooling2d (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_3 (Conv2D)	(None, 14, 14, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 64)	0
conv2d_4 (Conv2D)	(None, 7, 7, 128)	73,856
global_average_pooling2d (GlobalAveragePooling2D)	(None, 128)	0
feature_vector (Dense)	(None, 128)	16,512
dense_2 (Dense)	(None, 10)	1,290

Total params: 110,474 (431.54 KB)

Trainable params: 110,474 (431.54 KB)

Non-trainable params: 0 (0.00 B)

37. Train Feature Extractor

The feature extractor is trained as a Fashion-MNIST classifier.

The trained network provides a meaningful feature representation for real and generated Fashion-MNIST images.

Only the feature extractor is used after training; the classification output is not used for GAN training.

```
In [44]: # =====
# Section 37: Train Feature Extractor
# =====

# Compile the feature classifier
feature_classifier.compile(
    optimizer=keras.optimizers.Adam(
        learning_rate=0.001
    ),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# Train the classifier
feature_history = feature_classifier.fit(
    X_train,
```

```

y_train,
epochs=5,
batch_size=128,
validation_split=0.1,
verbose=1
)

```

Epoch 1/5

422/422 ————— **31s** 68ms/step - accuracy: 0.6331 - loss: 0.9985 - val_accuracy: 0.7415 - val_loss: 0.6735

Epoch 2/5

422/422 ————— **11353s** 27s/step - accuracy: 0.7617 - loss: 0.6286 - val_accuracy: 0.7825 - val_loss: 0.5796

Epoch 3/5

422/422 ————— **29s** 68ms/step - accuracy: 0.8012 - loss: 0.5370 - val_accuracy: 0.8147 - val_loss: 0.4997

Epoch 4/5

422/422 ————— **24s** 56ms/step - accuracy: 0.8275 - loss: 0.4731 - val_accuracy: 0.8375 - val_loss: 0.4485

Epoch 5/5

422/422 ————— **21s** 50ms/step - accuracy: 0.8454 - loss: 0.4253 - val_accuracy: 0.8503 - val_loss: 0.4168

38. Extract Real and Generated Feature Representations

The trained feature extractor is used to convert both real and generated images into compact numerical feature vectors.

These feature vectors represent visual characteristics learned from the Fashion-MNIST dataset.

The real and generated feature distributions are then compared.

```

In [45]: # =====
# Section 38: Extract Feature Vectors
# =====

# Extract features from real images
real_features = feature_extractor.predict(
    real_eval_images,
    batch_size=256,
    verbose=1
)

# Extract features from generated images
generated_features = feature_extractor.predict(
    generated_eval_images,
    batch_size=256,
    verbose=1
)

```

```
print("Real feature shape:", real_features.shape)
print("Generated feature shape:", generated_features.shape)
```

40/40 ————— 2s 41ms/step

40/40 ————— 1s 32ms/step

Real feature shape: (10000, 128)

Generated feature shape: (10000, 128)

39. Domain-Specific Fréchet Distance

The Fréchet distance is calculated between the feature distributions of real and generated Fashion-MNIST images.

The calculation uses:

- Mean feature vector of real images
- Covariance matrix of real features
- Mean feature vector of generated images
- Covariance matrix of generated features

A smaller distance indicates that the generated feature distribution is closer to the real-image feature distribution.

This project uses a Fashion-MNIST-trained feature extractor rather than the standard ImageNet Inception feature extractor. Therefore, the value is reported as a domain-specific FID-style metric and should not be directly compared with standard Inception-FID values from other studies.

```
In [46]: # =====
# Section 39: Calculate Domain-Specific Fréchet Distance
# =====

from scipy.linalg import sqrtm

def calculate_frechet_distance(real_features, generated_features):
    """
    Calculate the Fréchet distance between two feature distributions.
    """

    # Calculate feature means
    mu_real = np.mean(
        real_features,
        axis=0
    )

    mu_generated = np.mean(
        generated_features,
        axis=0
    )

    # Calculate covariance matrices
```

```

sigma_real = np.cov(
    real_features,
    rowvar=False
)

sigma_generated = np.cov(
    generated_features,
    rowvar=False
)

# Difference between means
mean_difference = mu_real - mu_generated

# Product of covariance matrices
covariance_product = (
    sigma_real @ sigma_generated
)

# Numerical stability
covariance_sqrt = sqrtm(
    covariance_product
)

# Handle small numerical imaginary components
if np.iscomplexobj(covariance_sqrt):
    covariance_sqrt = covariance_sqrt.real

# Fréchet distance
distance = (
    mean_difference @ mean_difference
    + np.trace(sigma_real)
    + np.trace(sigma_generated)
    - 2 * np.trace(covariance_sqrt)
)

return float(distance)

domain_fid = calculate_frechet_distance(
    real_features,
    generated_features
)

print("=" * 60)
print("DOMAIN-SPECIFIC FID-STYLE EVALUATION")
print("=" * 60)
print(f"Fréchet Feature Distance: {domain_fid:.4f}")

```

```

=====
DOMAIN-SPECIFIC FID-STYLE EVALUATION
=====
Fréchet Feature Distance: 0.8633

```

40. Generated Image Diversity Analysis

Visual quality alone does not determine the quality of a generative model.

A Generator may produce visually plausible images while generating highly similar samples.

Therefore, sample diversity is examined by measuring the variation between generated feature representations.

Higher feature variation indicates greater diversity among the generated samples, although diversity should always be interpreted together with image quality.

```
In [47]: # =====
# Section 40: Diversity Analysis
# =====

# Calculate standard deviation for each feature dimension
feature_std = np.std(
    generated_features,
    axis=0
)

# Average feature variation
average_feature_std = np.mean(
    feature_std
)

print("=" * 60)
print("GENERATED IMAGE DIVERSITY")
print("=" * 60)
print(
    f"Average feature standard deviation: "
    f"{average_feature_std:.6f}"
)
```

```
=====
GENERATED IMAGE DIVERSITY
=====
Average feature standard deviation: 0.781667
```

41. Nearest-Neighbor Analysis

Nearest-neighbor analysis compares generated samples with real Fashion-MNIST samples in the learned feature space.

This provides an additional qualitative check for similarity between generated and real samples.

Very close nearest neighbors do not automatically indicate memorization, but the analysis helps investigate whether generated samples contain substantial variation rather than being identical copies.

```
In [48]: # =====
# Section 41: Nearest-Neighbor Analysis
# =====

from sklearn.metrics import pairwise_distances

# Use a manageable subset for nearest-neighbor analysis
NN_SAMPLES = 1000

real_nn_features = real_features[:NN_SAMPLES]
generated_nn_features = generated_features[:NN_SAMPLES]

# Calculate pairwise distances
distances = pairwise_distances(
    generated_nn_features,
    real_nn_features,
    metric="euclidean"
)

# Find closest real sample for every generated sample
nearest_distances = np.min(
    distances,
    axis=1
)

print("=" * 60)
print("NEAREST-NEIGHBOR ANALYSIS")
print("=" * 60)

print(
    "Mean nearest-neighbor distance:",
    nearest_distances.mean()
)

print(
    "Minimum nearest-neighbor distance:",
    nearest_distances.min()
)

print(
    "Maximum nearest-neighbor distance:",
    nearest_distances.max()
)
```

```
=====
NEAREST-NEIGHBOR ANALYSIS
=====
Mean nearest-neighbor distance: 2.919826
Minimum nearest-neighbor distance: 0.6009294
Maximum nearest-neighbor distance: 6.286521
```

42. Nearest-Neighbor Distance Distribution

The distribution of nearest-neighbor distances is visualized to examine how closely generated samples are positioned to real Fashion-MNIST samples in the learned feature space.

The distribution provides additional information about the variation among generated samples and their similarity to real samples.

```
In [49]: # =====
# Section 42: Nearest-Neighbor Distance Distribution
# =====

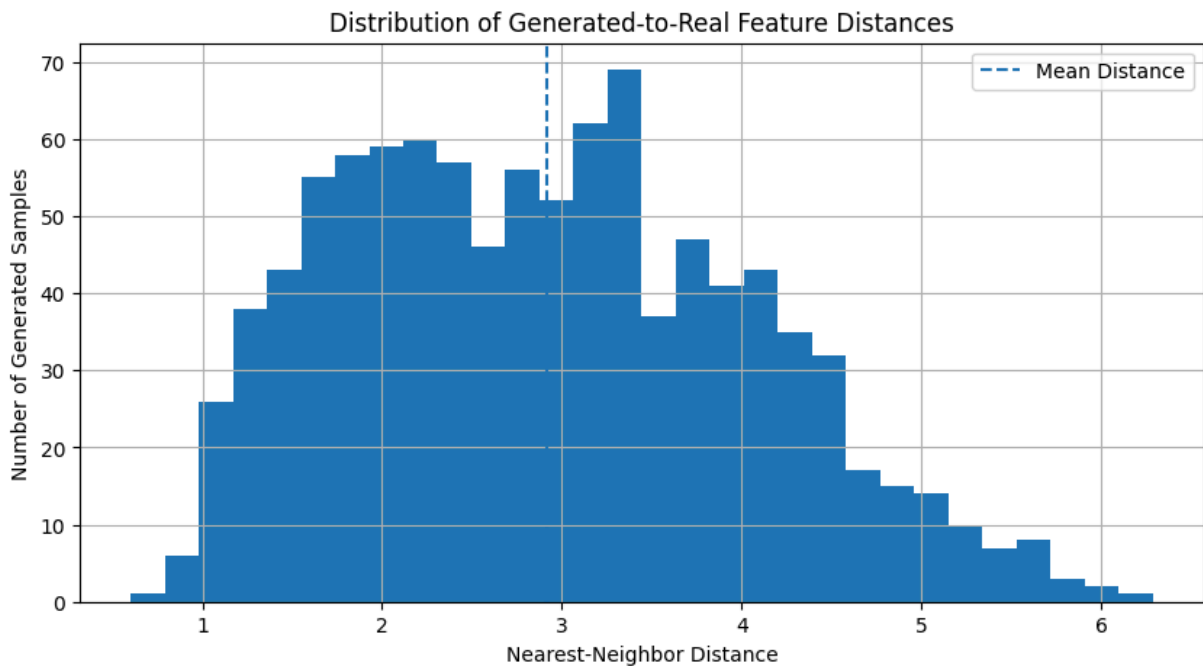
plt.figure(figsize=(10, 5))

plt.hist(
    nearest_distances,
    bins=30
)

plt.axvline(
    nearest_distances.mean(),
    linestyle="--",
    label="Mean Distance"
)

plt.xlabel("Nearest-Neighbor Distance")
plt.ylabel("Number of Generated Samples")
plt.title("Distribution of Generated-to-Real Feature Distances")
plt.legend()
plt.grid(True)

plt.show()
```



43. Summary of Quantitative Evaluation

The main quantitative evaluation metrics are collected into a single summary.

The reported metrics include the domain-specific Fréchet feature distance and nearest-neighbor statistics.

These values are used together with visual inspection and training-loss analysis to evaluate the trained DCGAN.

```
In [50]: # =====
# Section 43: Evaluation Metrics Summary
# =====

evaluation_results = {
    "Domain-Specific FID": domain_fid,
    "Mean NN Distance": nearest_distances.mean(),
    "Minimum NN Distance": nearest_distances.min(),
    "Maximum NN Distance": nearest_distances.max(),
    "Average Feature Std": average_feature_std
}

print("=" * 60)
print("DCGAN QUANTITATIVE EVALUATION SUMMARY")
print("=" * 60)

for metric, value in evaluation_results.items():
    print(f"{metric}: {value:.6f}")
```

```
=====
DCGAN QUANTITATIVE EVALUATION SUMMARY
=====
Domain-Specific FID: 0.863278
Mean NN Distance: 2.919826
Minimum NN Distance: 0.600929
Maximum NN Distance: 6.286521
Average Feature Std: 0.781667
```

```
In [51]: # =====
# Section 44: Save Evaluation Metrics
# =====

metrics_df = pd.DataFrame(
    {
        "Metric": list(evaluation_results.keys()),
        "Value": list(evaluation_results.values())
    }
)

metrics_df.to_csv(
    "results/evaluation_metrics.csv",
    index=False
)
```

```
print("Evaluation metrics saved successfully.")

display(metrics_df)
```

Evaluation metrics saved successfully.

	Metric	Value
0	Domain-Specific FID	0.863278
1	Mean NN Distance	2.919826
2	Minimum NN Distance	0.600929
3	Maximum NN Distance	6.286521
4	Average Feature Std	0.781667

45. Training Progress Visualization

Generated images from different training epochs are compared to visually examine how the Generator evolves during training.

The same fixed latent vectors were used during the progress visualization, allowing changes in generated outputs to be observed across training stages.

```
In [52]: # =====
# Section 45: Training Progress Visualization
# =====

progress_epochs = [
    1,
    5,
    10,
    15,
    20
]

plt.figure(figsize=(12, 10))

plot_position = 1

for epoch in progress_epochs:

    filename = f"generated_images/epoch_{epoch:03d}.png"

    if os.path.exists(filename):

        image = plt.imread(filename)

        plt.subplot(2, 3, plot_position)
        plt.imshow(image)
        plt.title(f"Epoch {epoch}")
```

```
plt.axis("off")

plot_position += 1

plt.suptitle(
    "DCGAN Training Progress",
    fontsize=16
)

plt.tight_layout()
plt.show()
```

DCGAN Training Progress



46. Final Generated Sample Grid

A final grid of synthetic Fashion-MNIST images is presented as the main qualitative result of the trained DCGAN.

The samples are generated from previously unseen random latent vectors.

```
In [53]: # =====
# Section 46: Final Generated Sample Grid
# =====

FINAL_SAMPLES = 36
```

```
final_noise_36 = tf.random.normal(  
    shape=(FINAL_SAMPLES, LATENT_DIM),  
    seed=SEED + 100  
)  
  
final_samples = generator(  
    final_noise_36,  
    training=False  
)  
  
plt.figure(figsize=(9, 9))  
  
for i in range(FINAL_SAMPLES):  
  
    plt.subplot(6, 6, i + 1)  
  
    image = (  
        final_samples[i, :, :, 0] + 1  
    ) / 2  
  
    plt.imshow(  
        image,  
        cmap="gray"  
    )  
  
    plt.axis("off")  
  
plt.suptitle(  
    "Final DCGAN Generated Fashion-MNIST Samples",  
    fontsize=16  
)  
  
plt.tight_layout()  
  
plt.savefig(  
    "results/final_generated_samples.png",  
    dpi=200,  
    bbox_inches="tight"  
)  
  
plt.show()  
plt.close()  
  
print("Final generated sample grid saved successfully.")
```

Final DCGAN Generated Fashion-MNIST Samples



Final generated sample grid saved successfully.

47. Final Model Evaluation Summary

The DCGAN was evaluated using multiple complementary approaches:

1. Training Loss Analysis

- Generator loss
- Discriminator loss
- Combined loss curves

2. Visual Evaluation

- Generated image samples
- Real versus generated comparison
- Training progression

3. Feature-Based Evaluation

- Domain-specific Fréchet feature distance
- Real and generated feature distributions

4. Diversity Analysis

- Feature standard deviation
- Variation among generated samples

5. Nearest-Neighbor Analysis

- Mean nearest-neighbor distance
- Minimum nearest-neighbor distance
- Maximum nearest-neighbor distance

No single metric is used as the only measure of GAN quality. The results are interpreted collectively because GAN losses, visual quality, distribution similarity, and sample diversity measure different aspects of generative performance.

48. Applications of GANs

Generative Adversarial Networks can be used in several applications where synthetic data or realistic generated content is required.

Examples include:

- Synthetic image generation
- Data augmentation
- Image-to-image translation
- Computer vision research
- Fashion and product design
- Medical image research
- Creative content generation
- Privacy-preserving synthetic data generation

In this project, the DCGAN is specifically used to generate synthetic Fashion-MNIST-style grayscale images.

49. Limitations

The implemented DCGAN has several limitations:

1. GAN training can be unstable because the Generator and Discriminator must learn simultaneously.

2. Generated image quality depends strongly on the architecture, hyperparameters, dataset, and number of training epochs.
3. Mode collapse can occur when the Generator produces similar samples instead of representing the full data distribution.
4. Fashion-MNIST contains relatively simple 28×28 grayscale images, so the results do not represent the challenges of high-resolution image generation.
5. The domain-specific Fréchet feature distance used in this project depends on the quality of the Fashion-MNIST feature extractor.
6. Quantitative metrics cannot completely replace human visual inspection of generated samples.

50. Future Scope

The project can be extended in several directions:

- Train the DCGAN for more epochs.
- Experiment with different Generator and Discriminator architectures.
- Tune learning rate, batch size, latent dimension, and optimizer parameters.
- Investigate techniques for reducing mode collapse.
- Compare DCGAN with other GAN architectures.
- Use conditional GANs to generate specific Fashion-MNIST categories.
- Use higher-resolution datasets for more challenging image generation.
- Explore additional feature-based evaluation methods.
- Compare generated samples across multiple training configurations.
- Investigate more advanced generative models such as StyleGAN and diffusion-based approaches.

51. Conclusion

In this project, a Deep Convolutional Generative Adversarial Network (DCGAN) was implemented using the Fashion-MNIST dataset.

The Generator learned to transform random latent vectors into synthetic 28×28 grayscale images, while the Discriminator learned to distinguish between real Fashion-MNIST images and generated samples.

The project included complete model training, generated-image visualization, Generator and Discriminator loss analysis, real-versus-generated comparison, feature-based evaluation, diversity analysis, and nearest-neighbor analysis.

The results demonstrate how adversarial training can be used to learn visual patterns from a dataset and generate new synthetic samples.

The evaluation was performed using multiple complementary approaches rather than relying on a single metric. This provides a broader view of the model's training behavior, generated image quality, feature similarity, and sample diversity.

52. Final Project Output Checklist

The completed project contains the following major components:

Dataset

- Fashion-MNIST dataset
- Data preprocessing
- Normalization to $[-1, 1]$

GAN Architecture

- DCGAN Generator
- DCGAN Discriminator
- Generator and Discriminator summaries

Training

- Generator loss
- Discriminator loss
- Adam optimizers
- Adversarial training loop
- Fixed-noise progress monitoring

Generated Results

- Generated image grids
- Real versus generated comparison
- Training progression

Quantitative Evaluation

- Pixel statistics
- Domain-specific Fréchet feature distance
- Feature diversity analysis
- Nearest-neighbor analysis

Saved Results

- Trained Generator
- Trained Discriminator
- Generated images
- Training loss graphs
- Real-versus-generated comparison
- Evaluation metrics CSV
- Final generated sample grid

Documentation

- Applications
- Limitations
- Future scope
- Conclusion